



- ① The Fundamentals of Java
- ② Object Oriented Programming
- ③ Data Structures and Algorithms

By Jawad Jammoul

Content:

Chapter 01 : Elementary Programming	Page
	1
Chapter 02 : Selections	6
Chapter 03 : Loops	8
Chapter 04 : Methods	11
Chapter 05 : Arrays	12
Chapter 06 : Objects and Classes	13
Chapter 07 : Strings	23
Chapter 08 : Inheritance and Polymorphism	28
Chapter 09 : ArrayList	34
Chapter 10 : Exception Handling and Text I/O	37
Chapter 11 : Abstract Classes and Interfaces	42
Chapter 12 : Inner and Nested Classes in Java	46
Chapter 13 : Enum	48
Chapter 14 : LinkedList	49
Chapter 15 : Stacks and Queues	55
Chapter 16 : Algorithms Analysis	59
Chapter 17 : Recursion	62
Chapter 18 : Tree	63
Chapter 19 : Graphs	76
Chapter 20 : HashTable & HashMap	86
Chapter 21 : Vector	88

Chapter 01 : Elementary Programming:

01*About Java:

What is Java?

Java is a high-level , object oriented programming language used to build cross-platform applications.

Why Using Java?

Java is a popular because it's platform-independent , secure , robust , and has a large community and rich libraries.

02*Definitions:

Class:

A class is a template or a set of instructions for creating something , it specifies what data it will handle and what operations can be performed on the data.

Main Method:

Is the entry points of any stand alone applications . It's where the program starts its execution .The main method is always defined inside the class.

```
public class className {  
    public static void main(String[] args) {  
        System.out.print("Text");  
    } //end main method  
} //end class
```

Output

Text

Attributes:

In the main method any method in Java , attributes (or variables) are used to store data and hold values that can be used and manipulated within the method.

```
dataType variableName ;  
variableName = value;  
//is Equivalent to  
dataType variableName = value;
```

```
public class className {  
    public static void main(String[] args) {  
        dataType variableName ;  
        variableName = value ;  
        System.out.print(variableName);  
    }  
}
```

Output

Value

03*Data Types and Variables:

DataTypes:

Primitive : int , double , long , float , boolean , char ...

Non-Primitive : String , Array , Structure , Classes ...

Variables:

Declaring:

int x ; // declare x to be an integer variable

double y ; //declare y to be a double variable

Assignment Statements:

x = 1 ; //Assign 1 to x

y = 5.3 ; //Assign 5.3 to y

You can store an int in a double , but not a double in an int without casting.

Example:

double x = 1 ; is Ok but int y = 3.2 ; is an error

You can add int and double into a double

You can't store the result of int+double in an int without casting

double x = 5;

int y = 3.2 ;

int a = 4;

double b = 2.5 ;

double c = a+b ;

int d = a + b;

int e = (int)(a+b) ; //ok

Caution!

int x = 12;

System.out.println("x") ; // print x (text)

System.out.println(x); //print 12 (value)

Concatenation operator(+):

```
int a,b,c ;
a= 7;
b= 3;
c =a+b;
System.out.println("Hey , the value of c is " + c);
```

Output
10

Final Modifier:

The final Modifier is used to define constants or to prevent modification , when a variable is declared as final , it's value cannot be changed once it's it has been initialized

```
final double a = 5.7;
double b = 1.3;
a= 1.3;
double c = a+ b;
System.out.print(c) ;
```

Output
Error

04*print Vs println:

print() outputs text without adding a new line at the end , while println() output text followed by a new line , moving the cursor to the next line.

```
System.out.print("Are you ");  
System.out.println("Ready!");  
System.out.print("for some ");  
System.out.println();  
System.out.println("Good Luck");
```

Output
Are you Ready! For some practice
Good Luck

05*Character(Char) & String:

Character:

.The character data type (char) is used to represent a single character

.A character literal is enclosed in single quotation marks:

Char variableName = 'value' ;

Example : char x = 'A' ;

ASCII Table	
Dec : Decimal value	
Char : Character	
Dec	Char
64	@
65	A
66	B
67	C
...	...
97	a
98	b

Caution:

'A' is a character but "A" is a String

Char casting:

char c = (char)97 ; -> a

int i = (int)'A' ; -> 65

Char with Scanner:

```
char c = scannerName.next().charAt(0);
```

String:

A String is a type of data that represents a sequence of characters , like words or sequence of characters , like word or sentences .It is used to manipulate text.

String variableName = "Text" ; //String name = "Jawad" ;

String With Scanner:

If it is word -> scannerName.next() ; //Jawad

If it is a sentence -> scannerName.nextLine() ; //Jawad Jammoul

\n and \t :

\n : is an escape sequence used to represent a new line character , which moves the cursor to the beginning of the next line.

\t : is an escape used to represent a horizontal tab character , which inserts a tab space in the output

Exercise 01
System.out.print("Java is");
System.out.print("/n fun");

Output
Java is
fun

Exercise 02
System.out.print("Java is /t fun");

Output
Java is fun

06*Errors:

Runtime errors:

A runtime error is an error that occurs while the program is executing , causing it to terminate unexpectedly due to illegal operations or invalid states . The division by zero is an illegal operation
System.out.print(1/0) ;

Logical errors:

Example 1:	
System.out.print(3*4-3) ; Output : 9	System.out.print(3*(4-3)); Output : 3
System.out.print(12/4+2); Output : 5	System.out.print(12/(4+2)); Output : 2

Syntax errors:

A syntax error is a mistake in the code's structure that prevents the program from compiling , typically due to incorrect use of language rules or punctuation

Incorrect : system.out.print(Welcome to Java")

Correct : System.out.print("Welcome to Java");

07*Scanner & Comments:

The Scanner class is used to read input from various sources like keyboard input , files , or streams , providing methods to parse and process the input data

Example
<pre>import java.util.Scanner; public class Name { public static void main(String[] args) { Scanner scannerName = new Scanner(System.in); System.out.println("Enter an integer:"); int a = scannerName.nextInt(); System.out.println("Enter a reel number:"); double b = scannerName.nextDouble(); } }</pre>

Comments:

Comments are used to annotate code , making it easier to understand for others or for yourself in the future.

Types:

[Single-line comments] : Start with // and continues to the end of the line

[Multi-line comments] : Start with /* and ends with */ It can span multiple lines

Example
//System.out.println("Good Morning"); System.out.println("Welcome to Java"); /* System.out.println("Let us start"); System.out.println("Have a nice day") ; */

Output
Welcome to Java

08*Numeric Operators:

Name	Meaning	Example	Result
+	Addition	34+1	35
-	Subtraction	34.0-0.1	33.9
*	Multiplication	300*30	9000
/	Division	1.0/2	0.5
%	Remainder	20%3	2

7485%10	5
7485%100	85
7485%1000	485

7485/10	748
7485/100	74
7485/1000	7

{ }	curly braces
[]	square brackets
' '	single quotation
" "	double quotation
,	comma
;	semi colon
.	Dot
:	colon

Increment	Decrement
x++ ; or ++x;	x--; or --x;
x=x+1;	x=x-1;
x+=1;	x-=1;

Arithmetic Expressions
(3+4x)/5 - 10(y-5)(a+b+c)/x + 9(4/x + 9+x)/y Is translated to (3+4*x)/5 - 10*(y-5)*(a+b+c)/x + 9*(4/x + 9+x)/y

5/2 yields an integer 2
5/2.0 or 5.0/2 yields a double value 2.5
5%2 yields 1
5/2 yields 2

09*Math Class:

a power b	Math.pow(a,b) ;
radical a	Math.sqrt(a);
Random nb between 0 and <1	Math.random();
PI = 3.141592653589793	Math.PI;

Math.Random() Arithmetic Expression

For example if you want a random nb between 3 and 10 :

```
int min = 3 ; int max = 10;  
double x = min + (int)(Math.random() * (max-min+1));
```

Chapter 02 : Selections

01*Boolean:

A boolean in Java is commonly used for conditional statements(selections) and control flow to determine the execution path based on logical conditions , it is a primitive data type that represents one of two possible values: true or false

Example: boolean x = true; boolean y = false;

boolean variableName = value;

02*Comparison operators & logical operators:

==	equal to
!=	not equal to
>	greater than
<	less than
>=	greater than or equal to
<=	less than or equal to

&&	And
	Or
!	Not
^	Xor

03*If-else:

One way if statements:

A one way if statement executes a block of code if a specified condition evaluates to true

```
if(condition) {  
    //code  
}
```

If an if statement executes a block of code only if a specified condition evaluates to true

```
if(condition)  
    statement;
```

same

```
if(condition) {  
    statement;  
}
```

if(true) : the code inside will always execute

if(false) : the code inside will never execute

Two-way if-else statements:

A two-way if-else statement provides a way to execute different code paths based on whether a condition evaluates to true or false

```
if(condition) {  
    //code1 (if true)  
}  
else {  
    //code2 (if false)  
}
```

Nested if-else:

A nested if-else statement is an if-else structure placed inside another if or else block to allow for multiple layers of conditional logic

```
if(condition1) {  
    //code1  
    if(condition1.1) {  
        //code1.1  
    }  
    else {  
        //code1.2  
    }  
}  
else {  
    //code2  
}
```

Alternating if-else:

An alternating if-else statement sequentially checks conditions and executes the corresponding block of the first true condition

```
if(condition1) {  
    //code1  
}  
else if(condition2) {  
    //code2  
}  
else if(condition3) {  
    //code3  
}  
else {  
    //code4  
}
```

04*Common Errors:

Common errors in if-else statements include missing braces , incorrect condition expressions , and improper nesting

if(condition); //code;	if(condition) { } //code;
Equivalent to	

05*Switch:

A switch statement is used to execute one block of code among many options based on the value of an expression

```
switch(expression) {  
    case value1:  
        //code1  
        break;  
    case value2:  
        //code2  
        break;  
    case value3:  
        //code3  
        break;  
    default :  
        //code4;  
        break;  
}
```

Chapter 03 : Loops

01*Why using loop?

Loops are used to repeatedly execute a block of code as long as a specified condition is true, making it efficient to handle repetitive tasks and iterate through collections or arrays

Example :

```
Write Hello World 100 Times  
System.out.println("Hello World!");  
System.out.println("Hello World!");  
System.out.println("Hello World!");  
System.out.println("Hello World!");  
....  
....  
System.out.println("Hello World!");  
System.out.println("Hello World!");
```

02*Loops:

while loop:

A while loop repeatedly executes a block of code as long as its specified condition remains true, allowing remains true , allowing for dynamic and flexible iterative based on changing conditions

```
while(condition) {  
    //code to be executed  
}
```

Same Example:

```
int i=0;  
while(i<100) {  
    System.out.println("Hello World!");  
    i++; //increment  
}
```

for loops:

A for loop provides a concise way to iterate over a range of values or a collection by initializing a counter, specifying a termination condition, and updating the counter on each iteration.

```
for(initialization ; condition ; update) {  
    //code to be executed  
}
```

Same Example:

```
for(int i=0 ; i<100 ; i++) {  
    System.out.print("Hello World");  
}
```

<pre>for(initialization;condition;update) { //one statement; }</pre>	<pre>for(initialization;condition;update) //one statement;</pre>
Equivalent to	

do-while loops:

A do-while loop executes a block of code once before checking its condition to execute the code as long as the condition remains true, ensuring that the code inside the loop runs at least once

```
do {  
    //code to be executed  
} while(condition);
```

Same Example:

```
int i=0;  
do {  
    System.out.println("Hello World");  
    i++;  
} while(i<100);
```

03*Difference between i++ and ++i :

++i increments the value of i before it is used in an expression, while i++ increments the value of i after it is used

Example:

```
int i=5,j=5;  
System.out.println("i : " + ++i);  
System.out.println("j : " + j++);  
System.out.println("Final i : " + i);  
System.out.println("Final j : " + j);
```

Output
i: 6
j: 5
Final i : 6
Final j : 6

04*Keywords break/continue:

break: The break Keyword is used to exit a loop prematurely , terminating the loop's execution and continuing with the code the follows it

boolean keep = true;

```
int i =0;
while(keep) {
    if(i==3)
        break;
    System.out.println(i);
    i++;
}
```

Output
0
1
2

continue: Skips the rest of the code inside the loop and jumps to the next iteration
It does not stop the loop , it just skips the current iteration

```
for(int i=1 ; i<=5 ; i++) {
    if(i==3) {
        continue; //skip when =3
    }
    System.out.println(i);
}
```

Output:
1
2
4
5

05*Nested Loops:

Nested loops are loops placed inside other loops , allowing for multiple levels of iteration over different ranges or conditions

Example:

```
for(int l=0;l<3;l++) {
    for(int j=0;j<3;j++) {
        System.out.print("(" +i+"," +j+ ") ");
    }
    System.out.println();
}
```

Chapter 04 : Methods

01*Why using methods?

A Method is a collection of statements that are grouped together to perform an operation

Using methods in Java helps organize code into reusable , manageable blocks , making programs easier to read , test , and maintain

02*Opening Problem with a method with a return value:

Opening Problem:

```
int sum = 0;
for(int i=0; i<=10 ; i++) {
    sum+=i;
    System.out.println("Sum from 1 to 10 is " +sum);
}
int sum = 0;
for(int i=20; i<=30 ; i++) {
    sum+=i;
    System.out.println("Sum from 20 to 30 is " +sum);
}
int sum = 0;
for(int i=35; i<=45 ; i++) {
    sum+=i;
    System.out.println("Sum from 35 to 45 is " +sum);
}
```

Solution:

```
public static int sumRange(int i1,int i2) {
    int sum=0;
    for(int i=i1 ; i<=i2 ; i++) {
        sum+=i;
    }
    return sum;
}
```

Invoke & Calling the Method:

```
public static void main(String[] args) {
    System.out.println("Sum from 1 to 10 is "+sumRange(1,10)); //here it called arguments
    System.out.println("Sum from 20 to 30 is " + sumRange(20,30));
    System.out.println("Sum from 35 to 45 is " + sumRange(35,45));
}
```

Output

```
Sum from 1 to 10 is 55
Sum from 20 to 30 is 275
Sum from 35 to 45 is 440
```

03*Void Method:

```
public class ProcedureSumRange {
    public static void sumRange(int i1,int i2) {
        int sum = 0;
        for(int i=i1 ; i<=i2 ; i++) {
            int sum+=i;
            System.out.println("Sum: "+sum);
        }
    }
    public static void main(String[] args) {
        sumRange(1,10);
        sumRange(20,30);
        sumRange(35,45);
    }
}
```

Output

```
Sum from 1 to 10 is 55
Sum from 20 to 30 is 275
Sum from 35 to 45 to 440
```

04*Method Signature:

Function Method:

```
public static returnType methodName(parameters) {
    //code
    return value;
}
```

Procedure Method:

```
public static void methodName(parameters) {
    //code
}
```

Chapter 05 : Arrays

01*Types of Arrays:

-->Single-Dimensional Array

-->Multidimensional Array

02*Single Dimensional Array:

A single-dimensional array is a collection of elements of the same data type stored in a single row

datatype[] variable = new Datatype[size];

datatype[] variable = {a,b,c,d};

```
public class Main {
    public static void main(String[] args) {
        int[] numbers = {10, 20, 30, 40, 50};
        for (int i = 0; i < numbers.length; i++) {
            System.out.println(numbers[i]);
        }
    }
}
```

Output
10
20
30
40
50

03*Multidimensional Array:

A multidimensional array is an array that contains other arrays. The most common type is a 2D array (rows and columns)

datatype[][] variable = new datatype[rowsize][columnsize];

**Datatype[][] variable = { {a,b,c} ,
 {d,e,f} ,
 {g,h,i}
 };**

```
public class Main {
    public static void main(String[] args) {
        int[][] matrix = {
            {1, 2, 3},
            {4, 5, 6},
            {7, 8, 9}
        };
        for (int row = 0; row < matrix.length; row++) {
            for (int col = 0; col < matrix[row].length; col++) {
                System.out.print(matrix[row][col] + " ");
            }
            System.out.println();
        }
    }
}
```

Output
1 2 3
4 5 6
7 8 9

Chapter 06 : Classes and Objects:

01*Definitions:

Classes:

A class is a blueprint or template used to create objects.

It defines the attributes (data) and methods (behavior) of objects.

Objects:

An object is an instance of a class.

It contains its own data and can perform actions defined by the class.

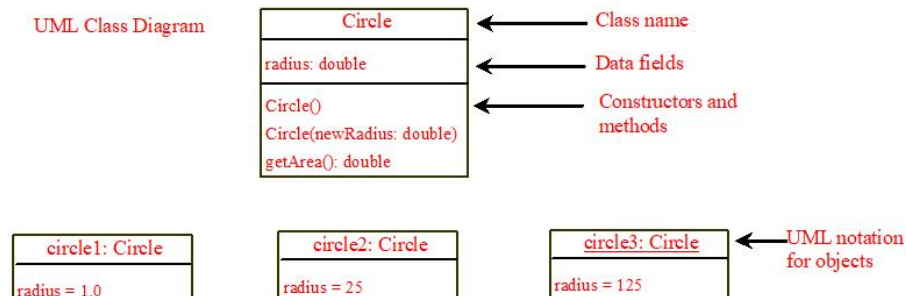
Constructor:

A constructor is a special method in a class used to initialize objects . It is called automatically when an object is created and often assigns initial values to the class fields

Garbage Collection:

Garbage Collection is the automatic process of removing unused objects from memory. It helps manage memory and prevents memory leaks in Java.

UML Class Diagram:



02*Creating an object:

```
className objectName = new className();
```

Or the other way is to write:

```
className objectName; //declaration statement
objectName = new className ; //creation statement
```

Example:

```
Circle myCircle1 = new Circle();
Circle myCircle2 = new Circle(5.0);
```

03*Accessing an Object's Data and Methods:

To reference the object's data we use the dot operator between the reference and the data field:

```
objectName.dataFieldName;
```

To invoke/call an object's method we use the dot operator between the reference and the method:

```
objectName.methodName(arguments);
```

04*Reference Data Fields and the null Value:

```
class Circle {
    int radius; // Default value is 0
    double area; // Default value is 0.0
    boolean isFilled; //Default value is false
    String color; //Default value is null
}
```

Hence null is the default value for any data field of a reference type
Note that null is a Keyword in Java

05*Using Classes from the Java Library:

The Date Class:

The Date class is found in the util package in the Java library To use we write

```
import java.util.Date;
public class Main {
    public static void main(String[] args) {
        Date date = new Date();
        System.out.println(date.toString());
    }
}
//Output: it displays a string like Mon Mar 23 20:50:19 EST 2020
```

java.util.Date	
+Date()	Constructs a Date object for the current time.
+Date(elapseTime: long)	Constructs a Date object for a given time in milliseconds elapsed since January 1, 1970, GMT.
+toString(): String	Returns a string representing the date and time.
+getTime(): long	Returns the number of milliseconds since January 1, 1970, GMT.
+setTime(elapseTime: long): void	Sets a new elapse time in the object.

The + sign indicates
public modifier

The Random Class:

The Random class found also in the util package is used to generate random values of different types

```
import java.util.Random();
Random r = new Random();
int x = r.nextInt(); // x could be any integer value
int y = r.nextInt(10); // y could be any integer value between 0 and 9
double z = r.nextDouble(); // z could be any double value between 0.0 inclusive and 1.0 exclusive
boolean flag = r.nextBoolean(); //flag could be either false or true
```

java.util.Random	
+Random()	Constructs a Random object with the current time as its seed.
+Random(seed: long)	Constructs a Random object with a specified seed.
+nextInt(): int	Returns a random int value.
+nextInt(n: int): int	Returns a random int value between 0 and n (exclusive).
+nextLong(): long	Returns a random long value.
+nextDouble(): double	Returns a random double value between 0.0 and 1.0 (exclusive).
+nextFloat(): float	Returns a random float value between 0.0F and 1.0F (exclusive).
+nextBoolean(): boolean	Returns a random boolean value.

06*Accessing Static vs Non-Static Attributes:

Static Attributes:

Static attributes belong to the class itself , not to individual objects

You can access static attributes directly using the class name without creating an object

Syntax: className.attribute;

Shared: All objects share the same static attribute value

Example:

```
class Circle {
    static double radius = 10.0;
}
class Main {
    public static void main(String[] args) {
        System.out.println("Static Radius: "+myCircle.radius);
    }
}
```

Non-Static Attributes:

Non-static attributes belong to individual objects of the class

To access non-static attributes , you must create an object of the class

Syntax: objectName.dataFieldName;

Unique: Each object has its own copy of the non-static attribute

Example:

```
class Circle {
    double radius;
}
class Main {
    public static void main(String[] args) {
        Circle myCircle1 = new Circle();
        myCircle1.radius = 5;
        System.out.println("Non-static Radius: "+myCircle1.radius);
    }
}
```

Static Constants:

Static Constants are final variables shared by all the instances of the class

Constants should be declared as a static final . For example , the constant PI in the Math class is defined as: static final double PI = 3.141592653589799323846;

Example:

```
public class Circle {
    double radius; //instance data (each Circle object has its own radius)
    static int numberOfObjects = 0; //static data(shared by all Circle objects)
    public Circle(double r) {
        radius = r;
        static int numberOfObjects = 0 ; //static data (shared by all Circle objects)
    }
    public Circle(double r) {
        radius = r;
        numberOfObjects++; // increase by 1 each time an object is created
    }
    public double getArea() { // instance method
        return radius*radius*Math.PI;
    }
}
```

```

public static int getNumberOfObjects() { //static method
    return numberOfObjects;
}
} //end class Circle

class TestCircle {
    public static void main(String[] args) {
        System.out.println("Before creating objects");
        System.out.println("The number of Circle objects is " + Circle.numberOfObjects);
        Circle c1 = new Circle(1);
        System.out.println("After creating c1");
        System.out.println("The number of Circle objects is " + c1.numberOfObjects);
        System.out.println("The number of Circle objects is " + Circle.numberOfObjects);
        Circle c2 = new Circle(5);
        System.out.println("After creating c2");
        System.out.println("The number of Circle object is " + c1.numberOfObjects);
        System.out.println("The number of Circle object is " + c2.numberOfObjects);
        System.out.println("The number of Circle object is " + Circle.numberOfObjects);
    }
}

```

Output
Before creating objects
The number of Circle objects is 0
After creating c1
The number of Circle objects is 1
The number of Circle object is 1
After creating c2
The number of Circle objects is 2
The number of Circle objects is 2
The number of Circle objects is 2

07*Array of Objects:

className[] arrayName = new className[arraySize]; //Declare and Create the array
arrayName[index] = new className(parameters); //initialize each element

Example Using Array of Circles:

```

import java.util.Random();
public class TestCircle {
    public static void main(String[] args) {
        Random r = new Random();
        Circle[] circleArray;
        for(int i=0;i<circleArray.length ; i++) {
            circleArray[i] = new Circle(r.nextDouble()*10);
            System.out.print("Circle " + i + " radius = " + circleArray[i].getRadius());
            System.out.println("area = " + circleArray[i].getArea());
        }
    }
}

```

Output	
Circle 0 radius = 5.323631308614646	area = 89.03603544910978
Circle 1 radius = 6.964428451073555	area = 152.37749675835462
Circle 2 radius = 3.6253233159062823	area = 41.28985531182978
Circle 3 radius = 7.25542582457795	area = 165.37721943564716
Circle 4 radius = 6.559096414885918	area = 135.15680048633092
Circle 5 radius = 5.8449836756578595	area = 107.32885044286944
Circle 6 radius = 7.4077945641189435	area = 172.39621729029602
Circle 7 radius = 7.515333962633156	area = 177.43792141377583
Circle 8 radius = 0.5383713110649391	area = 0.9105707398934727
Circle 9 radius = 0.9508741997827064	area = 2.8405078920179796

08*Visibility Modifiers:

Public : The member is accessible from anywhere in the program (within the same class + package, or from other packages)

Private : The member is accessible only within the same class where it is defined

Default : The member is accessible only within the same package

Note: public and private are Keywords

09*Data Field Encapsulation:

Why should data fields be private?

.To protect the integrity of the data by restricting direct access and modification

.Prevents unintended or invalid changes to the data fields from outside the class

The Need for Get and Set Methods:

Getter methods allow controlled , read-only access to private data fields

Setter methods allow controlled modification of private data fields , often including validation logic to ensure the data remains valid

```
class Circle {  
    private double radius;  
    public Circle(double newRadius) {  
        if(newRadius >=0)  
            radius = newRadius ;  
        else  
            radius = 0;  
    }  
    public double getRadius() {  
        return radius;  
    }  
    public void setRadius(double newRadius) {  
        if(newRadius >=0)  
            radius = newRadius;  
        else  
            radius = 0;  
    }  
    public double getArea() {  
        return radius*radius*Math.PI;  
    }  
}
```

10*Methods to Output The Data Fields:

The print() Method:

```
public void print() {  
    System.out.println("The radius is " + radius);  
    System.out.println("The number of objects is " + numberOfObjects);  
}
```

The toString() Method:

```
public String toString() {  
    String s = "The radius is " + radius + "\nThe number of objects is " + numberOfObjects;  
}
```

Or simply

```
public String toString() {  
    return "The radius is " + radius + "\nThe number of objects is " + numberOfObjects;  
}
```

Calling the toString() method:

```
public class TestCircle {  
    public static void main(String[] args) {  
        Circle c1 = new Circle(5.0);  
        System.out.println(c1.toString());  
        Circle c2 = new Circle(2.0);  
        System.out.println(c2.toString());  
    }  
}
```

Output

```
The radius is 5.0  
The number of objects is 1  
The radius is 2.0  
The number of objects is 2
```

11*Passing Objects to Methods:

Passing by Reference : Instead of copying the object , the memory address is sent to the method . Therefore , changes made to the object inside the method are reflected on the original object.

Example:

```
class Circle {  
    private double radius;  
    public Circle(double r) {  
        radius = r;  
    }  
    public double getRadius() {  
        return radius;  
    }  
    public void setRadius(double r) {  
        if(r>0)  
            radius = r;  
        else  
            System.out.println("Invalid radius");  
    }  
}  
  
public class Main {  
    public static void main(String[] args) {  
        Circle myCircle = new Circle(5.0);  
        System.out.println("Before : " + myCircle.getRadius());  
        changeRadius(myCircle);  
        System.out.println("After : " + myCircle.getRadius());  
    }  
    public static void changeRadius(Circle c) {  
        c.setRadius(10.0);  
    }  
}
```

Output

```
Before : 5.0  
After : 10.0
```

12*The Scope of a Variable:

Local Scope :

Declared inside methods , constructors ,or blocks , accessible only within that block

Instance Scope :

Declared in a class but outside methods , belongs to an object , and accessible by all methods of the class

Class /Static Scope:

Declared as static in a class , shared across all objects , and accessed without creating an object

```
public class ScopeExample {  
    int instanceVar = 10; //Instance scope  
    static int staticVar = 20; //Static scope  
    public void show() {  
        int localVar = 30; //Local scope  
    }  
}
```

What if we do not use the this Keyword when we should?

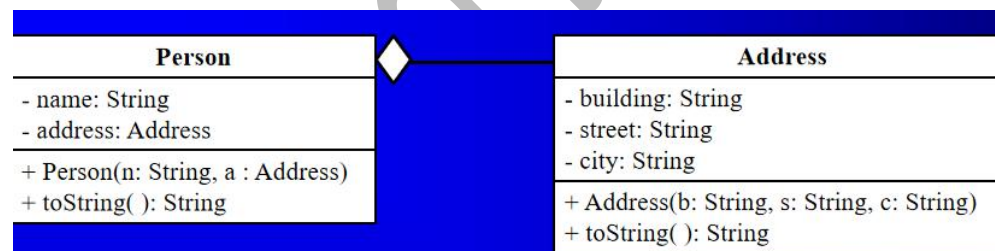
```
public class Foo {  
    private int i = 5;  
  
    public void setI(int i) {  
        i = i;  
    }  
}
```



Suppose that f1 is an object of Foo.

Invoking f1.setI(10) will not initialize the data field i to 10. It will instead set the parameter i to itself (i.e. change the parameter i from 10 to 10).

13*Object Composition:



```
class Person {  
    private String name;  
    private Address address; //This is the statement that creates the composition relationship  
    public String toString() {  
        return "name = " + name + " address = " + address.toString();  
    }  
}  
  
class Address {  
    private String building;  
    private String street;  
    private String city;  
}  
  
public Address(String b,String s , String c) {  
    building = b;  
    street = s;  
    city = c;  
}
```

```

    public String toString() {
        return building + ", " + street + ", " + city;
    }
}
public class Test {
    public static void main(String[] args) {
        Address a = new Address("Building A", "Hamra Stree", "Beirut");
        Person p = new Person("Jawad", a);
        System.out.println(p);
    }
}

```

14*Enhanced For Loop (for-each):

The enhanced for loop is a simplified version of the for loop to iterate through arrays or collections.
 "For each element of this array/collection , do the following."

Syntax:

```

for(Type element : collectionOrArray) {
    //use element
}

```

Traditional for loop:

```

int[] arr = {1, 2, 3, 4};
for(int i = 0; i < arr.length; i++) {
    System.out.println(arr[i]);
}

```

Enhanced for-each loop:

```

int[] arr = {1,2,3,4};
for(int num: arr) {
    System.out.println(num);
}

```

Output
1
2
3
4

Examples with collections:

```

import java.util.ArrayList;
public class TestForEach {
    public static void main(String[] args) {
        ArrayList<String> names = new ArrayList<>();
        names.add("Ali");
        names.add("Sara");
        names.add("John");
        for(String name : names) {
            System.out.println(name);
        }
    }
}

```

Output
Ali
Sara
John

Example : iterating over an Array of Objects

```
class Student {
    String name;
    int age;
    Student(String name , int age) {
        this.name = name;
        this.age = age;
    }
    void display() {
        System.out.println(name + " is " + age + " years old");
    }
}

public class TestForEachObject {
    public static void main(String[] args) {
        Student[] students = {
            new Student("Ali",20) ,
            new Student("Sara" , 22) ,
            new Student("John" , 19);
        };
        //Enhanced for loop (for-each) over objects
        for (Student s : students) {
            s.display(); //call method on each object
        }
    }
}
```

Output:

```
Ali is 20 years old
Sara is 22 years old
John is 19 years old
```

Example: Iterating over a Collection of Objects

```
import java.util.ArrayList;
public class TestForEachObjectList {
    public static void main(String[] args) {
        ArrayList<Student> students = new ArrayList<>();
        students.add(new Student("Ali" , 20));
        students.add(new Student("Sara" ,22));
        students.add(new Student("John" , 19));
        //Enhanced for loop over ArrayList of objects
        for(Student s : students) {
            s.display();
        }
    }
}
```

Output

```
Ali is 20 years old
Sara is 22 years old
John is 19 years old
```

Chapter 07 : Strings

01*Constructing a String:

String :

Sequence of Characters

Using the constructor:

```
String s = new String("Text");  
String s = new String("Hello World");
```

Using a String literal:

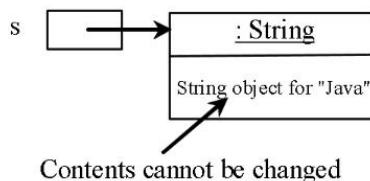
```
String s = "Text";  
String s = "Hello World";
```

Using an array of characters:

The following statement creates the String "Jawad";

```
char k = { 'J', 'a', 'w', 'a', 'd' };  
String text = new String(k);
```

After executing `String s = "Java";`



02*Internal Strings:

JVM uses a unique instance for string literals with the same character sequence to improve efficiency and save memory. Such an instance is called an interned string. For example, the following statements :

```
String s1 = "Welcome to Java";  
String s2 = new String("Welcome to Java");  
String s3 = "Welcome to Java";  
System.out.println("s1 == s2 is " + (s1 == s2)); //s1 == s2 is false  
System.out.println("s1 == s3 is " + (s1 == s3)); //s1 == s3 is false
```

03*String Comparison:

equals() : is used to compare the contents of strings

```
String s1 = new String("Welcome");  
String s2 = "Welcome";  
String s3 = "Hi";  
if(s1.equals(s2)) {  
    //checks if s1 and s2 have the same contents  
    //For the above example it gives true  
}  
if(s1.equals(s3)) {  
    //checks if s1 and s3 have the same contents  
    //For the above example it gives false  
}
```

== : is used to compare the string references

```
String s1 = new String("Welcome");
String s2 = "Welcome" ;
String s3 = s1;
if(s1 == s2) {
    //checks if s1 and s2 have the same reference
    //For the above example it gives false
}
if(s1 == s3) {
    //checks if s1 and s3 have the same reference
    //For the above example it gives true
}
```

equalsIgnoreCase() : Ignores the letter case (lower versus upper case) when comparing

```
String s1 = new String("Welcome");
String s2 = "Welcome" ;
if(s1.equals(s2))
    //gives false since w is different from W
if(s1.equalsIgnoreCase(s2))
    //gives true since w and W are considered the same
```

compareTo() : Returns an integer greater than 0 , equal to 0 , or less than 0 to indicate whether this string is greater than , equal to , or less than s1

```
String s1 = new String("Welcome");
String s2 = "Welcome" ;
if(s1.compareTo(s2) > 0) //is s1 greater than s2?
if(s1.compareTo(s2) < 0) //is s1 smaller than s2?
if(s1.compareTo(s2) == 0) // do s1 and s2 have the same content?
String s1 = "abc" , s2 = "abc" ;
String s3 = "aba" , s4 = "abd" ;
int x;
x = s1.compareTo(s2);
//x = 0 since s1 and s2 are equal
x = s1.compareTo(s3);
// x > 0 since s1 > s3 because 'c' is > 'a' ('c' = 99 & 'a' = 97)
// See ASCII Table
x = s1.compareTo(s4);
//x < 0 since s1 < s4 because 'c' is < 'd' ( 'c' = 99 & 'd' = 100)
```

compareToIgnoreCase(): Same as compareTo except that the comparison is case-insensitive

```
String s1 = "abc" , s2 = "ABC" ;
int x;
x = s1.compareTo(s2);
//x > 0 since s1 > s2 because 'a' > 'A' ('a' = 97 , 'A' = 65)
x = s1.compareToIgnoreCase(s2) ;
//x = 0 since s1 and s2 are equal when the case is ignored
```

regionMatches() : This method compares portions of two strings for equality

```
String s1 = "scde" , s2 = "abcdef";
if(s1.regionMatches(1,s2,2,3)
/*1 is the starting index in s1, and 2 is the starting index in s2 , 3 is the length or the number of characters to be compared*/
```

startsWith(prefix) : to check whether string starts with a specified prefix

endsWith(suffix) : to check whether string ends with a specified suffix

```
String s = "howtodoinJava.com";  
if(s.startsWith("how")) //gives true  
if(s.startsWith("howl")) //gives false  
if(s.endsWith("com")) //gives true  
if(s.endsWith("dom")) //gives false
```

04*String length , Characters , and Combining Strings:

length() : Returns the number of characters in this string

```
String p = "Hello World";  
int len = p.length(); //len = 11
```

charAt() : Returns the character at the specified index from this string

```
String m = "Hello";  
for(int i=0;i<m.length();i++)  
System.out.println(m.charAt(i));
```

Output
H
e
l
l
o

concat() : Returns a new string that concatenate this string with string s1

```
String s1 = "Welcome" , s2 = " to Java";  
String s3 = s1.concat(s2);  
s3 will contain the string "Welcome to Java"
```

05*Obtaining Substrings:

Indices	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14
message	W	e	l	c	o	m	e		t	o		J	a	v	a

message.charAt(0) message.length() is 15 message.charAt(14)

substring(beginIndex) : Returns this string's substring that begins with the character at the specified begin Index and extends to the end of the string

```
String s1 = "Welcome to Java" ;
```

To extract the substring from index 11 till the end we write:

```
String s2 = s1.substring(11);
```

substring(beginIndex , endIndex) : Returns the string's substring that begins at the specified

beginIndex and extends to the character at the index endIndex -1

```
String s1 = "Welcome to Java";  
String s1 = s1.substring(0,11); // Welcome to
```

06*Converting , Replacing , and Splitting Strings:

toLowerCase() : Returns a new string with all characters converted to lowercase .

"Welcome".toLowerCase() returns a new string , welcome

toUpperCase() : Returns a new string with all character converted to uppercase.

"Welcome".toUpperCase() returns a new string , WELCOME

trim() : Returns a new string with blank characters trimmed on both sides.

" Welcome to Java ".trim() returns a new string, Welcome to Java

replace() : Returns a new string that that replaces all matching character in this string with the new character.

"Welcome".replace('e','A') returns a new String . WAlcomA

replaceFirst() : Return a new string that replaces the first matching substring in this string with the new substring.

"Welcome".replace("e","AB") returns a new String. WABlcome

replaceAll() : Returns a new string that replaces all matching substrings in this string with the new substring.

"Welcome".replaceAll("e","AB") returns a new String , WABlcomAB

replace(oldString , newString) : Works exactly like replaceAll.

"Welcome".replace("e","AB") returns a new String , WABlcomAB

07*Splitting a String:

The split method can be used to extract tokens from a string with the specified delimiters . For example , in the following code the delimiter is the string "#"

split(delimiter) : Returns an array of strings consisting of the substrings split by the delimiter.

```
String s = "Java#HTML#Perl" ;  
String[] tokens = s.split("#");  
For(int i=0; i<tokens.length ;i++)  
System.out.print(tokens[i]+" ");
```

The delimiter # is not stored in the array tokens(an array of strings)

```
String s = "cdabefabgh" ;  
String[] tokens = s.split("ab");  
For(int i=0 ;i<tokens.length;i++)  
System.out.print(tokens[i] + " ");
```

08*The "." delimiter:

If want to split a string using the delimiter dot "." such as

```
String a = "a.jpg";  
String[] str = a.split(".");
```

The split does not work because "." is a reserved character

Instead, we should use the following statement:

```
String[] str = a.split("\\.");
```

09*Finding a Character or a substring in a String:

indexOf(ch : char) : int Returns the index of the first occurrence of ch in the string. Returns -1 if not matches

```
"Welcome to Java".indexOf("W") returns 0
```

indexOf(ch: char , fromIndex: int): int Returns the index of the first occurrence of ch after fromIndex in the string. Returns -1 if not matches

```
"Welcome to Java".indexOf('x') returns -1
```

indexOf(s: String): int Returns the index of the first occurrence of string s in this string. Returns -1 if not matches

```
"Welcome to Java".indexOf('o' , 5) returns 9
```

indexOf(s : String , fromIndex: int): int Returns the index of the first occurrence of string s in this string after fromIndex.. Returns -1 if not matches

```
"Welcome to Java".indexOf("come") returns 3
```

lastIndexOf(ch : int): int Returns the index of the last occurrence of ch in this string. Returns -1 if not matches

```
"Welcome to Java".indexOf("Java" , 5) returns 11
```

lastIndexOfch : int , fromIndex : int) : int Returns the index of the last occurrence of ch before fromIndex in this string. Returns -1 if not matches

```
"Welcome to Java".indexOf("java" , 5) returns -1
```

lastIndexOf(s : String) : int Returns the index of the last occurrence of string s . Returns -1 if not matches

```
"Welcome to Java".lastIndexOf('a') returns 14
```

valueOf(c: char) : String Returns a string considering of the character C

```
char c = 'A' ;  
String s = String.valueOf(c);  
System.out.println(s);
```

```
Output
```

```
A
```

valueOf(data : char[]) : String Returns a string considering of the characters in the array

```
char[] data = {'J' , 'a' , 'v' , 'a'};  
String s = String.valueOf(data);  
System.out.println(s)
```

```
Output
```

```
Java
```

valueOf(d : double) : String Returns a string representing the double value

```
double d = 3.14;  
String s = valueOf(d);  
System.out.println(s)
```

```
Output
```

```
3.14
```

valueOf(f : float) : String

Returns a string representing the float value

```
float f = 2.5f;  
String s = String.valueOf(f);  
System.out.println(s)
```

Output

2.5

valueOf(i : int) : String

Returns a string representing the long value

```
int i =10;  
String s = String.valueOf(i);  
System.out.println(s);
```

Output

10

valueOf(b : boolean) : String

Returns a string representing the boolean value

```
boolean b = true;  
String s = String.valueOf(b);  
System.out.println(s);
```

Output

true

10*Converting Strings to Integer and Double using Integer.parseInt() and Double.parseDouble() :

To convert a String into an integer we use the method Integer.parseInt() and into a double we use the method Double.parseDouble().

```
String s1 = "12" , s2 = "12.5" ;  
int n = Integer.parseInt(s1);  
double m = Double.parseDouble(s2);
```

As a result n will contain the integer value 12 and m will contain the double value 12.5

Note that if s1 and s2 contain non digit characters then an error will occur. Of course s2 can contain the dot .

Chapter 08 : Inheritance and Polymorphism

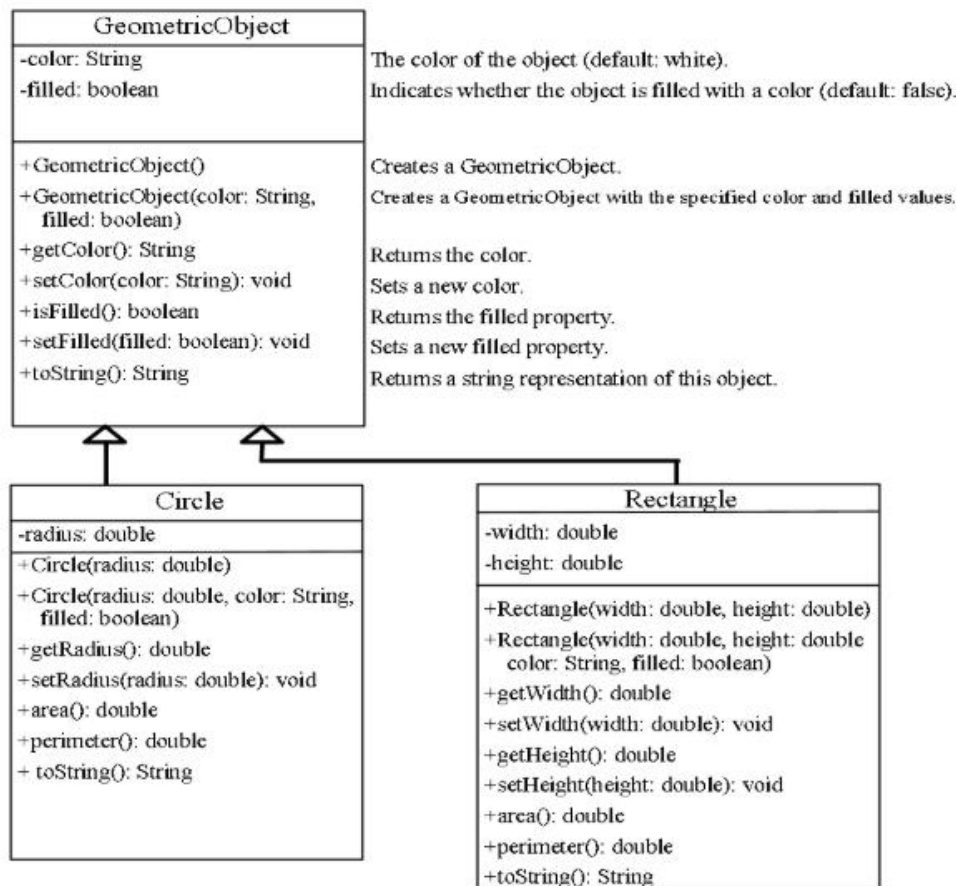
01*Inheritance Definition:

Inheritance: is a mechanism where one class(child/subclass) acquires the properties and behaviors of another class(parent/superclass)

02*Super classes and Subclasses:

A superclass is the parent class that provides its properties and methods to other classes , serving as a base for inheritance . A subclass is the child class that inherits the features of the superclass and can also define its own additional properties and behaviors

03*Geometric Object Class Application:



```

class GeometricObject {
    private String color;
    private boolean filled;
    public GeometricObject() {
        color = "white";
        filled = false;
    }
    public GeometricObject(String c, boolean f) {
        color = c;
        filled = f;
    }
    public String getColor() {
        return color;
    }
    public void setColor(String c) {
        color = c;
    }
    public boolean isFilled() {
        return filled;
    }
    public void setFilled(boolean f) {
        filled = f;
    }
    public String toString() {
        String s = "The color is " + color;
        if(filled)
            s+= ".\nThe geometric object is filled";
        else
            s+= ".\nThe geometric object is not filled";
        return s;
    }
}
    
```

```

} //end class GeometricObject

class Circle extends GeometricObject {
    private double radius;
    public Circle(String c , boolean f , double r) {
        super(c,f); //calls the constructor of GeometricObject using the keyword super
        radius = r;
    }
    public double getRadius() {
        return radius;
    }
    public void setRadius(double radius) {
        this.radius = radius;
    }
    public double area() {
        return radius*radius*Math.PI;
    }
    public double perimeter() {
        return 2 * radius *Math.PI;
    }
    public String toString() {
        //call toString of GeometricObject using the keyword super
        return super.toString() + ".\nThe radius is "+radius;
    }
} //end class circle

class Rectangle extends GeometricObject {
    private double width;
    private double height;
    public Rectangle(String c , boolean f , double width , double height) {
        super(c,f); //calls constructor of GeometricObject
        this.width = width;
        this.height = height;
    }
    public double getWidth() {
        return width;
    }
    public void setWidth(double width) {
        this.width = width;
    }
    public double getHeight() {
        return height;
    }
    public void setHeight(double height) {
        this.height = height ;
    }
    public double area() {
        return width*height;
    }
    public double perimeter() {
        return 2 * (width+height);
    }
    public String toString() {
        //calls toString of GeometricObject
        return super.toString() + ".\nThe height is " + height + ".\nThe width is " + width;
    }
} //end class Rectangle

public class TestingInheritance {
    public static void main(String[] args) {
        Circle c1 = new Circle("red" , true ,2);
        System.out.println(c1.getColor());
        System.out.println(c1.getRadius());
        System.out.println(c1);
        GeometricObject g1 = new GeometricObject("black" , false);
    }
}

```

```

System.out.println(g1.getColor());
//System.out.println(g1.getRadius());
// getRadius() will give an error since class GeometricObject does not have a method called getRadius()
System.out.println(g1);
Rectangle r1 = new Rectangle("green" , true , 3 ,4);
System.out.println(r1.getColor());
System.out.println(r1.getHeight());
System.out.println(r1);
}
}

```

04*Using the super Keyword:

The super keyword is used to call a superclass's constructor or access its methods and fields

It can be used in two ways:

-> To call superclass constructor

-> To call a superclass method

super() :

Invokes the no-arg constructor of its superclass

super(arguments):

Invokes the super class constructor that matches the arguments

05*Example on the Impact of a Super class without no-arg constructor:

What is the error ?

```

public class Fruit {
    System.out.println("Fruit's constructor is invoked");
}
}
public class Apple extends Fruit {
    public Apple(String name) {
    }
}
}

```

The constructor of Apple does not contain an explicit call to the superclass Fruit constructor so it will call the no-arg constructor of Fruit which does not exist. This will cause an error

06*Overriding Methods:

Method overriding is when a subclass provides a specific implementation of a method that is already defined in its superclass, with the same method signature

An instance(non-static) method can be overridden only if it is accessible. Thus a private method cannot be overridden, because it is not accessible outside its own class

Like an instance method, a static method can be inherited. However, a static method cannot be overridden.

```

class Circle extends GeometricObject {
    //Other methods are omitted
    @override
    public String toString() {
        return super.toString() + "/nradius is " + radius;
    }
}
}

```

Example:

```
class Animal {
    public void sound() {
        System.out.println("Animal makes a sound");
    }
}
class Dog extends Animal {
    @override
    public void sound() {
        System.out.println("Dogs barks");
    }
}
public class Main {
    public static void main(String[] args) {
        Animal myAnimal = new Animal();
        Animal myDog = new Dog();
        myAnimal.sound();
        myDog.sound();
    }
}
```

07*Overloading Methods:

Method overloading is the ability to define multiple methods with the same name but different parameter lists(number or type) within the same class

A.If the type of the parameters is different:

```
public static int max(int num1 , int num2) {
    //body
}
public static double max(int num1 , int num2) {
    //body
}
```

B.If the number of the parameters is different:

```
public static int sum(int num1 , int num2) {
    //body
}
public static int sum(int num1 , int num2 , int num3) {
    //body
}
```

C.If the order of the parameters is different:

```
public static void method(int num , char ch) {
    //body
}
public static void method(char ch , int num) {
    //body
}
```

What if we said?

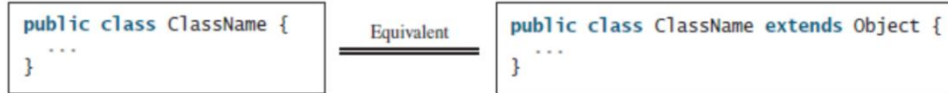
```
public static int max(int n1 , int n2) {
    //body
}
public static double max(int n1 , int n2) {
    //body
}
```

The code you provided will result in a compilation error because method overloading requires methods to differ in parameters , but not just in return type

08*The object class and its toString() Method:

.The object class itself is found in the java.lang package , and every class in Java implicitly extends it unless it specifically extends another class

.Every class in Java is a subclass of the class Object found in the java library in the lang package
If no inheritance is specified when a class is defined , the superclass of the class is Object by default . For example , the following two class definitions are the same



09*Polymorphism:

Polymorphism is one of the core concepts of Object-Oriented Programming(OOP) . It allows an object to take on many forms . In Java , it means that a single reference variable of a superclass type can refer to objects of different subclass types , and the appropriate method is called at runtime

Example:

```
class Animal {  
    void sound() {  
        System.out.println("Some generic animal sound");  
    }  
}  
class Dog extends Animal {  
    void sound() {  
        System.out.println("Woof");  
    }  
}  
class Cat extends Animal {  
    void sound() {  
        System.out.println("Meow!");  
    }  
}  
public class Main {  
    public static void main(String[] args) {  
        Animal a1 = new Dog() ;  
        Animal a2 = new Cat();  
        a1.sound(); // Woof!  
        a2.sound(); //Meow!  
    }  
}
```

10*Casting Objects:

Means converting one type of reference to another within the same inheritance hierarchy , allows us to treat an object as a more specific or more general type

Example:

```
Animal a = new Dog(); //Upcasting  
((Dog)a).sound(); //Downcasting to call Dog's specific method  
Animal a = new Dog(); //Upcasting  
Dog d = (Dog) a; //Downcasting (valid)  
d.sound();
```

The instanceof Operator:

The instanceof operator checks whether an object is an instance of a specific class or subclass
It helps prevent ClassCastException during downcasting

Example:

```
Animal a = new Cat();
if(a instanceof Dog) {
    Dog d = (Dog)a;
    d.sound();
}
else {
    System.out.println("Not a Dog object");
}
```

Output

Not a Dog object

Chapter 09 : ArrayList

01*The ArrayList class:

The ArrayList class is a resizable array from the java.util package , which allows dynamic addition and removal of elements . Unlike regular arrays , its size adjusts automatically as elements are added or removed , we **import java.util.ArrayList;**

02*The ArrayList Methods:

ArrayList() : Creates an empty ArrayList

```
ArrayList<String> names = new ArrayList<>();
System.out.println(names);
```

Output

[]

add(): Adds an element to the list
Adds an element to the list

```
names.add("Ali");
names.add("Sara");
System.out.println(names);
```

Output

[Ali,Sara]

clear(): Removes all elements

```
names.clear();
System.out.println(names);
```

Output

[]

contains() : Checks if element exists

```
System.out.println(names.contains("Ali"));
System.out.println(names.contains("Khaled"));
```

Output
true
false

get() : Returns element at index

```
System.out.println(names.get[1]);
```

Output
Sara

indexOf() : First occurrence index

```
System.out.println(names.indexOf("Ali"));
```

Output
0

lastIndexOf():Last occurrence index

```
System.out.println(names.lastIndexOf("Ali"));
```

Output
3

remove(int index) : Removes element by index

```
names.remove(2);
System.out.println(names);
```

Output
[Ali , Sara ,Ali]

remove(object o) : Removes first matching element

```
names.remove("Ali");
System.out.println(names);
```

Output
[Sara , Omar]

size(): Returns number of elements

```
System.out.println(name.size());
```

Output
4

set(): Replaces element at index

```
names.set(1,"Lina");
System.out.println(name);
```

Output
[Ali , Lina , Omar , Ali]

03*The ArrayList class as a generic class:

ArrayList<Type> list = new ArrayList<>();

Examples:

```
ArrayList<String> cities = new ArrayList<String>();  
ArrayList<Date> dates = new ArrayList<Date>();
```

04*Non-Generic versus Generic ArrayList:

Generic ArrayList:

Can store only one type of objects(specified using <Type>

Type-safe ? Then no Casting Needed

Errors are caught at compile-time

Example:

```
import java.util.ArrayList;  
public class Main {  
    public static void main(String[] args) {  
        ArrayList<String> list = new ArrayList<>(); //Generic  
        list.add("Java");  
        list.add("Python");  
        //list.add(10); will cause a compile-error time  
        String s = list.get(0);  
        System.out.println(s);  
    }  
}
```

Output

Java

Non-Generic ArrayList :

Can store any type of objects

Not type-safe -> you need casting when retrieving elements

Errors are caught at runtime , not compile-time

Example:

```
import java.util.ArrayList ;  
public class Main {  
    public static void main(String[] args) {  
        ArrayList list = new ArrayList(); //Non-generic  
        list.add("Java");  
        list.add(10);  
        list.add(3.5);  
        String s = (String) list.get() ;  
        System.out.println(s);  
        //String x = (String) list.get(1); //Runtime error  
    }  
}
```

Output

Java

Chapter 10 : Exception Handling and Text I/O :

01*Introduction:

When a program runs into a runtime error(also known as exception) , the program terminates abnormally . How can you handle the runtime error so that the program can continue to run or terminate gracefully ? this is the subject we will introduce in this chapter

02*Exception Handling Overview:

Without Exception Handling:

```
public Main {
    public static void main(String[] args) {
        int a = 10 ;
        int b = 0;
        int result = a/b;
        System.out.println("Result: " + result);
    }
}
```

Output:

Exception in thread "main" java.lang.ArithmeticException: /by zero at Main.main(Main.java: 5)

Without Exception Handling:

```
public class Main {
    public static void main(String[] args) {
        try {
            int a = 10;
            int b = 0;
            int result = a/b;
            System.out.println("Result "+ result) ;
        }
        catch(ArithmeticException e) {
            System.out.println("Error: Cannot divide by zero.");
        }
    }
}
```

Output

Error: Cannot divide by zero

Fixing it without using Exception Handling:

```
public class Main {
    public static void main(String[] args) {
        int a = 10;
        int b = 0;
        if(b!=0) {
            int result = a/b;
            System.out.println("The quotient is: " + quotient);
        }
        else {
            System.out.println("Error: Cannot divide by zero");
        }
    }
}
```

Output

Error: Cannot divide by zero

03*Throwing and catching an Exception:

What is try-catch?

The try-catch block in Java is a mechanism to handle exceptions(runtime errors)

Try block: Contains the code that might throw an exception

Catch block: Catches and handles the exception if it occurs , preventing the program from crashing

What is the Value Thrown ?

When an exception occurs , Java creates an exception object that contains:

3.2.1.The type exception (e.g. , ArithmeticException , NullPointerException)

3.2.2.A message describing the error: The value thrown is the exception object , which contains details about the error and can be accessed in the catch block

04*Some of Java's Important Exceptions:

ArithmeticException	Division by zero or some other arithmetic problem
ArrayIndexOutOfBoundsException	An array index is less than zero or greater than or equal to the array's length
FileNotFoundException	Reference to a file that cannot be found
NumberFormatException	Use of an illegal number format , as when calling a method
StringIndexOutOfBoundsException	A String index is less than zero or greater than or equal to the String's length

NullPointerException: Reference to an object that has not been instantiated

```
String name = null;  
System.out.println(name.length());
```

Output

Exception in thread "main" java.lang.NullPointerException

indexOutOfBoundsException: An array of String index is out of bounds

```
int[] number = {1,2,3};  
System.out.println(number[5]);
```

Output

Exception in thread "main" java.lang.ArrayIndexOutOfBoundsException: Index 5 out of bounds of length 3

Handling NullPointerException :

```
public class Main {  
    public static void main(String[] args) {  
        String name = null;  
        if(name != null)  
            System.out.println(name.length());  
        else  
            System.out.println("The object is null , can't call methods on it");  
    }  
}
```

Handling IndexOutOfBoundsException :

```
public class Main {  
    public static void main(String[] args) {  
        int[] numbers = {1,2,3};  
        int index = 5;  
        if(index>=0 && index<numbers.length)  
            System.out.println(numbers[index]);  
        else  
            System.out.println("Index out of bounds: " + index);  
    }  
}
```

Output
Index out of bounds: 5

InputMismatchException:

```
import java.util.Scanner;
public class Main {
    public static void main(String[] args) {
        Scanner scan = new Scanner(System.in);
        System.out.println("Enter a number: ");
        try {
            int number = scanner.nextInt();
            System.out.println("You entered : "+ number);
        }
        catch(InputMismatchException e) {
            System.out.println("Invalid input ! Please enter an integer");
        }
    }
}
```

IllegalArgumentException: Calling a method with an improper argument

```
public class Main {
    public static void setAge(int age) {
        if(age<0) {
            throw new IllegalArgumentException("Age cannot be negative!");
        }
        System.out.println("Age is: "+ age);
    }
    public static void main(String[] args) {
        setAge(25); //valid
        setAge(-5); // Throw IllegalArgumentException
    }
}
```

Output
Age is: 25 Exception in thread "main" java.lang.IllegalArgumentException: Age cannot be negative!

Checked Exceptions vs Unchecked Exceptions:

Checked Exception:

- A checked exception is an exception that is checked at compile-time
- The compiler forces you to handle it using try-catch or throws
- Usually occurs in operations outside your control (like file I/O , network , database)

Example:

```
import java.io.File;
import java.io.FileNotFoundException;
import java.util.Scanner;
public class CheckedExample {
    public static void main(String[] args) {
        try {
            File file = new File("data.txt");
            Scanner scanner = new Scanner(System.in);
        }
    }
}
//Here ,FileNotFoundException is a checked exception , the compiler won't let you ignore it
```

Unchecked Exception:

- An unchecked exception is an exception that occurs at runtime
- The compiler does not check whether you handle it or not
- Usually occurs due to programming mistakes (like null pointer , divide by zero , array index out of bounds).

Example:

```
public class UncheckedExample {
    public static void main(String[] args) {
        int[] numbers = {1,2,3};
        System.out.println(numbers[5]); //ArrayIndexOutOfBoundsException
    }
}
```

05*The Exception Class:

The Exception class in Java is the base class for all checked exceptions , representing errors that can be handled by the program

Example:

```
public class Main {
    public static void divide(int a , int b) {
        try {
            if(b==0) {
                throw new Exception("Division by zero is not allowed!");
            }
            System.out.println("Result: "+(a/b));
        }
        catch(Exception e) {
            System.out.println("Caught Exception: " + e.getMessage());
        }
    }
    public static void main(String[] args) {
        divide(10,2); //Valid division
        divide(10,0); //Throws Exception
    }
}
```

Declaring Exceptions like:

--> public void myMethod() throws IOException

--> public void myMethods() throw IOException , OtherException

06*Throwing Exceptions:

Throwing exceptions is the process of using the throw keyword to explicitly signal an error during program execution

Example:

```
public class Main {
    public static void checkNumber(int number) {
        if(number<0) {
            throw new IllegalArgumentException("Number cannot be negative!");
        }
        System.out.println("Valid number : " + number);
    }
    public static void main(String[] args) {
        checkNumber(5); //valid
        checkNumber(-3); //Throws exception
    }
}
```

07*File Class:

What is File class?

```
import java.io.File;
```

Can be used to:

Check if a file/directory exists

Create new files or directories

Delete files

Get file properties(name, path , size , readability , writability)

Creating a File Object:

```
File f1 = new File("data.txt"); //file in project folder
File f2 = new File("C:/Users/Jawad/Documents/file.txt"); //absolute path
File f3 = new File("C:/Users/Jawad/Documents", "file.txt"); //parent + child
```

Common Methods with Examples and Output:

Check if file exists:

```
File f1 = new File("data.txt");
if(f1.exists()) {
    System.out.println("File exists!");
}
else {
    System.out.println("File does not exists!");
}
```

Example Output (if file exists) :

```
File exists!
```

Example Output (if file doesn't exists):

```
File does not exist!
```

Check if it's a file or directory:

```
File f1 = new File("example.txt");
if(f1.isFile()) {
    System.out.println("This is a file");
}
if(f1.isDirectory()) {
    System.out.println("This is a directory");
}
```

Example output (data.text is a file):

```
This is a file
```

Create a new file:

```
import java.io.IOException;
try {
    if(f1.createNewFile()) {
        System.out.println("File created successfully");
    }
    else {
        System.out.println("File already exists");
    }
}
catch(IOException e) {
    e.printStackTrace();
}
```

Example Output (if file already exists) :

File already exists

Example Output (if file does not exists):

File created successfully

Delete a file:

```
File f1 = new File("example.txt");
if(f1.delete()) {
    System.out.println("File deleted successfully");
}
else {
    System.out.println("File to delete file");
}
```

Example Output (if deletion successful):

File deleted successfully

Get file properties:

```
File f1 = new File("example.txt");
System.out.println("File name: " + f1.getName());
System.out.println("Path: " + f1.getPath());
System.out.println("Absolute path: " + f1.getAbsolutePath());
System.out.println("Readable : " + f1.canRead());
System.out.println("Writable: " + f1.canWrite());
System.out.println("File size in bytes: " + f1.length());
```

Example Output:

```
File name: example.txt
Path : example.txt
Absolute path : C:\Users\Jawad\project\example.txt
Readable : true
Writable : true
File size in bytes : 0
```

Chapter 11 : Abstract Classes and Interfaces

01*Abstract Classes:

Abstract classes are classes that cannot be instantiated directly and are declared using the abstract keyword. They are used to provide a common blueprint for subclasses , containing both implemented methods are abstract methods that must be overridden by subclasses . While objects cannot be created directly from an abstract class , objects can be created from subclasses that extends it , inheriting its properties and behavior

Note : the use of keyword abstract in the class header and the methods header

02*Abstract Class GeometricObject:

```
public abstract class GeometricObject {
    private String color;
    private boolean filled;
    public GeometricObject(String c , boolean f) {
        color = c;
        filled = f;
    }
    public String getColor() {
        return color;
    }
    public void setColor(String c) {
        color = c;
    }
    public boolean isFilled() {
        return filled;
    }
    public String toString() {
        String s = "The color is " + color ;
        if(filled)
            s+= ".\nThe geometric object is filled" ;
        else
            s+= "\nThe geometric object isn't filled" ;
        return s;
    }
    public abstract double area() ;
    public abstract double perimeter() ;
}
```

03*Why Abstract Methods?

Abstract methods are used when a method's behavior must be defined by subclasses , ensuring that each subclass provides its specific implementation . They serve as a contract that enforces consistency across all subclasses inheriting from the abstract class

Example:

```
abstract class Shape {
    abstract double calculateArea();
}
class Circle extends Shape {
    double radius;
    Circle(double radius) {
        this.radius = radius;
    }
    @Override
    double calculateArea() {
        return Math.PI*radius*radius ;
    }
}
class Rectangle extends Shape {
    double width , height ;
    rectangle(double width , double height) {
        this.width = width ;
        this.height = height ;
    }
    @Override
    double calculateArea() {
        return width*height ;
    }
}
```

```

public class Main {
    public static void main(String[] args) {
        Shape c = new Circle(5);
        Shape r = new Rectangle(4,6);
        System.out.println("Circle Area: " + c.calculateArea());
        System.out.println("Rectangle Area: " + r.calculateArea());
    }
}

```

Output

```

Circle Area : 78.53981633974483
Rectangle Area : 24.0

```

04*Interesting points on Abstract classes:

.Abstract classes are like regular classes , but you cannot create instances of abstract classes using the new operator . Hence , the following statement is not allowed :

```
GeometricObject g = new GeometricObject("While" , true) ;
```

.An abstract method is defined without implementation . Its implementation is provided by the subclasses . If a subclass of an abstract superclass doesn't implement all the abstract methods , the subclass extended from an abstract class , all the abstract methods must be implemented

.A class that contains abstract methods must be abstract . However , it is possible to define an abstract an abstract class that contains no abstract methods

.An abstract method cannot be contained in a nonabstract(concrete class)

05*Interfaces:

What is an interface and why is an interface useful?

An interface is a class like construct that contains only constants and abstract methods . In many ways , an interface is similar to an abstract class , but the intent of an interface is to specify a behavior for objects . For example , you can specify that the objects are comparable using the appropriate interface

An interface is treated like a special class in Java . Like an abstract class , you cannot create an instance from an interface using a new operator , but in the most cases you can use an interface more or less the same way you use an abstract class .For example , you can use an interface as a data type for a variable

Define an Interface:

```

public interface interfaceName {
    /*Describe how to eat*/
    public abstract String howToEat() ;
}

```

Example:

```

public interface Edible {
    public abstract String howToEat() ;
}
abstract class Animal {
}
class Tiger extends Animal {
}
abstract class Fruit implements Edible {
}
class Chicken extends Animal implements Edible {
    public String howToEat() {

```

```

        return "Chicken : Fry it" ;
    }
}
class Apple extends Fruit {
    public String howtoEat() {
        return "Apple : Make apple cider " ;
    }
}
class Orange extends Fruit {
    public String howToEat() {
        return "Orange : Make orange juice " ;
    }
}
public class TestEdible {
    public static void main(String[] args) {
        Edible[] edibles = {
            new Orange() , new Chicken() , new Apple()
        };
        for(int i = 0 ; i<edibles.length ; i++) {
            //if(edibles[i] instanceof Edible)
            System.out.println(edibles[i].howToEat());
        }
    }
}

```

06*Omititng Modifiers in Interfaces:

<pre> public interface T1 { public static final int k = 1 ; public abstract void p() ; } </pre>	<pre> public interface T1 { int k = 1 ; void p() ; } </pre>
Equivalent	

07*The comparable Interface:

Package java.lang:

```

public interface Comparable<E> {
    public int compareTo(E o);
}

```

Application:

```

public class Application {
    public static void main(String[] args) {
        Circle c1 = new Circle(1);
        Circle c2 = new Circle(2);
        Circle c3 = new Circle(3);
        int x = c1.compareTo(c2) ;
        System.out.println("Comparing c1 and c2:");
        if(x==0)
            System.out.println("The two circles are equals") ;
        else if(x<0)
            System.out.println("The first circle is smaller");
        else
            System.out.println("The first circle is larger");
        if(c2.compareTo(c3) == 0)
            System.out.println("The two circles are equal");
        else if(c2.compareTo(c3) <0)
            System.out.println("The first circle is smaller");
        else
            System.out.println("The first circle is larger");
    }
}

```

```
}  
}
```

Chapter 12 : Inner and Nested Classes in Java

01*Definition:

A nested class is a class defined inside another class

If the nested class is non-static , it is called and increase encapsulation

Nested classes help logically group classes that are only used in one place and increase encapsulation

02*Types of Nested Classes:

A)Member Inner class:

Declared inside another class but outside methods

Needs an object of the outer class to be created.

Example:

```
class Outer {  
    private String msg = "Hello from Outer" ;  
    class Inner {  
        void show() {  
            System.out.println(msg) ; // can access private members  
        }  
    }  
}  
public class TestInner {  
    public static void main(String[] args) {  
        Outer outer = new Outer();  
        Outer.Inner inner = outer.new Inner();  
        inner.show();  
    }  
}
```

Output
Hello from Outer

b)Static Nested Class:

Declared static inside another class.

Can be created without an instance of the outer class

Can only access static members of the outer class

Example:

```
class Outer {  
    static String msg = "Hello from Static Nested Class" ;  
    static class Inner {  
        void show() {  
            System.out.println(msg);  
        }  
    }  
}  
public class TestStaticNested {  
    public static void main(String[] args) {  
        Outer.Inner inner = new Outer.Inner() ;  
        inner.show();  
    }  
}
```

Output
Hello from Static Nested Class

c)Local Inner Class:

Declared inside a method

Its scope is limited to that method

Example:

```
class Outer {  
    void display() {  
        class LocalInner {  
            void msg() {  
                System.out.println("Hello from Local Inner Class");  
            }  
        }  
        LocalInner local = new LocalInner();  
        local.msg();  
    }  
}  
public class TestLocalInner {  
    public static void main(String[] args) {  
        Outer outer = new Outer() ;  
        outer.display();  
    }  
}
```

Output

Hello from Local Inner Class

d)Anonymous Inner Class:

A class without a name

Used for short implementations , often with interfaces or abstract classes

Example:

```
abstract class Animal {  
    abstract void sound() ;  
}  
public class TestAnonymousInner {  
    public static void main(String[] args) {  
        Animal a = new Animal() {  
            void sound() {  
                System.out.println("Dog barks");  
            }  
        };  
        a.sound();  
    }  
}
```

Output

Dog barks

Chapter 13: Enum

01*Definition:

An Enum (short for enumeration) is a special Java type used to define a fixed set of constants Useful when you have a limited list of possible values (like days of the week , sizes , directions).

02*Basic Example:

```
enum Size {  
    SMALL , MEDIUM , LARGE  
}  
public class TestEnum {  
    public static void main(String[] args) {  
        Size s = Size.MEDIUM ;  
        System.out.println("Selected size:" + s);  
    }  
}
```

Output

Selected size : MEDIUM

03*Enum with Fields and Methods:

```
enum Size {  
    SMALL(1) , MEDIUM(2) , LARGE(3) ;  
    private int value;  
    Size(int value) {  
        this.value = value;  
    }  
    int getValue() {  
        return value;  
    }  
}  
public class TestEnumAdvanced {  
    public static void main(String[] args) {  
        Size s = Size.LARGE ;  
        System.out.println("Size: "+s+" , Value : "+s.getValue());  
    }  
}
```

Output

Size: LARGE , Value: 3

Translated to a Class :

```
class Size {  
    public static final Size SMALL = new Size(1);  
    public static final Size MEDIUM = new Size(2);  
    public static final Size LARGE = new Size(3);  
    private int value;  
    private Size(int value) {  
        this.value = value;  
    }  
    public int getValue() {  
        return value;  
    }  
}  
public class TestSizeClass {  
    public static void main(String[] args) {  
        Size s = Size.LARGE;  
        System.out.println("Size: " + s + " , Value: " + s.getValue());  
    }  
}
```

```
}  
}
```

Output

Size : Size@<hashcode>, Value: 3

Chapter 14: LinkedList

01*Definition:

A LinkedList is a linear data structure where each element (called a node) points to the next one. It is a dynamic structure, meaning it can grow or shrink in size during runtime.

02*Why using LinkedList:

Dynamic size: Unlike arrays, it doesn't need a fixed size.

Efficient insertions / deletions: Adding or removing nodes is faster (especially in the middle or beginning).

No memory waste. Uses memory only as needed (no pre-allocated size).

03*Each Node Consists of Two Parts:

Data part: Stores the actual value (e.g., int, String, etc.)

Link (Reference) part: Points to the next node in the list.

04*NodeData Class:

```
class NodeData {  
    private int value;  
    public NodeData(int value) {  
        this.value = value;  
    }  
    public int getValue() {  
        return value;  
    }  
    public void setValue(int value) {  
        this.value = value;  
    }  
    public int compareTo(NodeData x) {  
        if(this.value > x.value)  
            return 1;  
        else if(this.value < x.value)  
            return -1;  
        else  
            return 0;  
    }  
}
```

05*Node Class:

```
public class Node {
    private NodeData data;
    private Node next;
    public Node(NodeData data) {
        this.data = data;
        this.next = null;
    }
    public NodeData getData() {
        return data;
    }
    public void setData(NodeData data) {
        this.data = data;
    }
    public Node getNext() {
        return next;
    }
    public void setNext(Node next) {
        this.next = next;
    }
}
```

06*LinkedList Class:

```
public class LinkedList {
    private Node head;
    //Constructor
    public LinkedList() {
        this.head = null;
    }
    //Getter
    public Node getHead() {
        return head;
    }
    //Setter
    public void setHead(Node head) {
        this.head = head;
    }
}
```

print() : Used to display all elements in the list in order

```
public void print() {
    Node current = head;
    while(current!=null) {
        System.out.print(current.getData().getValue() + "->");
        current = current.getNext();
    }
    System.out.println("null");
}
```

isEmpty() : Checks if the list is empty

```
public boolean isEmpty() {
    return head == null;
}
```

countNodes() : Returns the total number of nodes in the list

```
public int countNodes() {
    int count = 0;
    Node current = head;
    while(current!=null) {
        count++;
        current = current.getNext();
    }
    return count;
}
```

belongs(NodeData data) : Checks if a specific value exists in the list

```
public boolean belongs(NodeData data) {
    Node current = head;
    while(current != null) {
        if(current.getData().compareTo(data) == 0) {
            return true;
        }
        current = current.getNext();
    }
    return false;
}
```

addHead(NodeData data) : Adds a new node at the beginning of the list

```
public void addHead(NodeData data) {
    Node newNode = new Node(data);
    newNode.setNext(head);
    head = newNode;
}
```

addTail(NodeData data) : Adds a new node at the end of the list

```
public void addTail(NodeData data) {
    Node newNode = new Node(data);
    if(head == null) {
        head = newNode;
        return;
    }
    Node current = head;
    while(current.getNext() != null) {
        current = current.getNext();
    }
    current.setNext(newNode);
}
```

insertPosition(NodeData data , int pos): Inserts a new node at a specific position in the list

```
public void insertPosition(NodeData data , int pos) {
    if(pos<=0) {
        addHead(data);
        return;
    }
    Node current = head;
    int index = 0;
    while(current!=null && index<pos-1) {
        current = current.getNext();
        index++;
    }
    Node newNode = new Node(data);
    newNode.setNext(current.getNext());
    current.setNext(newNode);
}
```

equals(LinkedList other) : Compares two lists to check if they are equal in structure and data

```
public boolean equals(LinkedList other) {
    Node current1 = this.head;
    Node current2 = other.head;
    while(current1 != null && current2 != null) {
        if(current1.getData().compareTo(current2.getData()) != 0) {
            return false;
        }
        current1 = current1.getNext();
        current2 = current2.getNext();
    }
    return current1 == null && current2 == null ;
}
```

addImplace(LinkedList other): Appends another list to the end of the current list

```
public void addImplace(LinkedList other) {
    if(this.head == null)
        this.head = other.head;
    else {
        Node current = this.head;
        while(current.getNext() != null) {
            current = current.getNext();
        }
        current.setNext(other.head);
    }
}
```

copy() : Creates and returns a new list that is a copy of the current one

```
public LinkedList copy() {
    LinkedList newList = new LinkedList();
    if(head == null)
        return newList;
    Node current = head;
    while(current != null) {
        newList.addTail(new NodeData(current.getData().getValue()));
        current = current.getNext();
    }
    return newList;
}
```

reverse() : Reverses the order of the nodes in the list

```
public void reverse() {
    Node prev = null ;
    Node current = head;
    while(current != null) {
        Node next = current.getNext() ;
        current.setNext(prev);
        prev = current ;
        current = next;
    }
    head = prev;
}
```

isPal() : Checks if the list is a palindrome (same forward and backward)

```
public boolean isPal() {
    LinkedList reversed = this.copy();
    reversed.reverse() ;
    return this.equals(reversed);
}
```

merges(LinkedList other) : Merges two sorted lists into a new sorted list

```
public LinkedList merges(LinkedList other) {
    LinkedList result = new LinkedList();
    Node p1 = this.head;
    Node p2 = other.head;
    while(p1 != null && p2 != null) {
        if(p1.getData().compareTo(p2.getData())<=0) {
            result.addTail(new NodeData(p1.getData().getValue()));
            p1 = p1.getNext();
        }
        else {
            result.addTail(new NodeData(p2.getData().getValue()));
            p2 = p2.getNext();
        }
    }
    while(p1 != null) {
        result.addTail(new NodeData(p1.getData().getValue()));
        p1 = p1.getNext();
    }
    while(p2 != null) {
        result.addTail(new NodeData(p2.getData().getValue()));
        p2 = p2.getNext();
    }
    return result;
}
```

deletePosition(int pos) : Deletes the node at a specific position

```
public void deletePosition(int pos) {
    if(pos<0 || head == null)
        return;
    if(pos == 0) {
        deleteHead();
        return;
    }
    Node current = head;
    for(int i =0 ; current.getNext() != null && i< pos-1 ; i++){
        current = current.getNext();
    }
    if(current.getNext() != null) {
        current.setNext(current.getNext().getNext());
    }
}
```

deleteHead() : Removes the first node of the list

```
public void deleteHead() {
    if(head != null) {
        head = head.getNext();
    }
}
```

deleteTail() : Removes the last node of the list

```
public void deleteTail() {
    if(head == null || head.getNext() == null) {
        head = null;
        return;
    }
    Node current = head;
    while(current.getNext().getNext() != null) {
        current = current.getNext();
    }
    current.setNext(null);
}
```

07*Application class:

```
public class Application {
    public static void main(String[] args) {
        //Create first LinkedList
        LinkedList list1 = new LinkedList();
        list1.addHead(new NodeData(10));
        list1.addTail(new NodeData(20));
        list1.addTail(new NodeData(30));
        list1.insertPosition(new NodeData(25), 2);
        list1.print();
        list1.deleteTail();
        list1.print();
        list1.deletePosition(1);
        list1.print();
        list1.reverse();
        list1.print();
        LinkedList copyList = list1.copy();
        System.out.println("Copy equals original ?" + list1.equals(copyList)); //true
        list1.addTail(new NodeData(10));
        list1.addTail(new NodeData(20));
        list1.print();
        System.out.println("Is Palindrome?" + list1.isPal());
        LinkedList list2 = new LinkedList();
        list2.addTail(new NodeData(5));
        list2.addTail(new NodeData(15));
        copyList.addImplace(list2);
        copyList.print();
        LinkedList sorted1 = new LinkedList();
        sorted1.addTail(new NodeData(1));
        sorted2.addTail(new NodeData(3));
        LinkedList sorted2 = new LinkedList();
        sorted2.addTail(new NodeData(2));
        sorted2.addTail(new NodeData(4));
        LinkedList merged = sorted1.merges(sorted2);
        System.out.print("Merged List: ");
        merges.print();
    }
}
```

Output: 10 -> 20 -> 25 -> 30 -> null isEmpty: false Count : 4 Belongs 25? true 20 -> 25 -> null; 20 -> null 20 -> null Copy equals original ? true 20 -> 5 -> 15 -> null Merged List : 1 -> 2 -> 3 -> 4 -> null

Chapter 15 : Stack and Queues

01*ADT:

An Abstract Data Type (ADT) is a specification of a collection of data and the operations (methods) that can be performed on it

The definition of an ADT only mentions what operations are to performed but not how these operations will implemented. The process of providing only the essentials and hiding the details is known as abstraction

A data structure is the implementation for an ADT

02*Stack Array Implementation:

push(int value) : add an item to the top of the stack

pop() : remove an item from the top of the stack

top() or peek() : returns the top item without removing it

isEmpty() : returns true if the stack is empty

isFull() : checks whether the stack has reached its maximum size

```

class Stack {
    private int[] arr;
    private int top;
    public Stack(int size) {
        arr = new int[size];
        top = -1;
    }
    public void push(int value) {
        if(isFull()) {
            System.out.println("Stack is empty");
            return -1;
        }
        else
            return arr[top--];
    }
}
public int top() {
    if(isEmpty()) {
        System.out.println("Stack is empty");
        return -1;
    }
    else
        return arr[top];
}
public boolean isEmpty() {
    return top == -1 ;
}
public boolean isFull() {
    return top == arr.length - 1;
}
}

```

03*Stack LinkedList:

```
public class Stack {
    private Node top;
    public Stack() {
        top = null;
    }
    public boolean isEmpty() {
        return top == null;
    }
    public void push(NodeData item) {
        Node newNode = new Node(item);
        newNode.next = top;
        top = newNode;
    }
    public NodeData pop() {
        if(isEmpty()) {
            System.out.println("Stack is empty");
            return null;
        }
        else {
            NodeData value = top.data;
            top = top.next;
            return value;
        }
    }
    public NodeData top() {
        if(isEmpty()) {
            System.out.println("Stack is empty");
            return null;
        }
        else {
            return top.data;
        }
    }
}

class Node {
    NodeData data;
    Node next;
    public Node(NodeData data) {
        this.data = data;
        this.next = null;
    }
}
```

Advantage of use Stack LinkedList:

- No fixed size limit , it can grow dynamically as needed.
- Faster insertion and deletion at the top (O(1) time).
- No need to manage array resizing or shifting elements.
- Efficient memory usage since no unused space is allocated.
- Easier to implement recursion-like behavior with linked nodes.
- Can store complex data types using custom NodeData objects.

04*Implementing a queue using an array:

A queue is a waiting line or a linear list in which items are added at one end and deleted from the other end.

Familiar examples are queues at a bank , a supermarket , etc. People are supposed to join the queue at the tail and exit from the head

Queue , as the name suggests is used whenever we need to manage any group of objects in an order in which the first one coming in , also gets out first , also gets out first (First In First Out or simply FIFO) while the others wait for their turn

These are the basic operations we want to perform on a queue:

- Add an item to the queue (we say enqueue)
- Take an item off the queue (we say dequeue)
- Check whether the queue is empty
- Inspect the item at the head of the queue

enqueue(int item) : Adds an element to the end of the queue if it's not full

dequeue() : removes and returns the front element of the queue

isEmpty() : Checks if there are no elements in the queue

isFull() : Checks if the queue has reached its maximum capacity

```
class Queue {
    private int[] queueArray ;
    private int head , tail ;
    public Queue() {
        this(10);
    }
    public Queue(int size) {
        queueArray = new int[size];
        head = 0;
        tail = -1;
    }
    public void enqueue(int item) {
        if(isFull()) {
            System.out.println("Queue is full");
        }
        else {
            queueArray[++tail] = items ;
        }
    }
    public int dequeue() {
        if(isEmpty()) {
            System.out.println("Queue is empty");
            return -1;
        }
        else {
            return queueArray[head++];
        }
    }
    public boolean isEmpty() {
        return head>tail ;
    }
    public boolean isFull() {
        return tail == queueArray.length - 1;
    }
    public int head() {
        if(isEmpty()) {
            System.out.println("Queue is empty");
            return -1;
        }
        else {
            return queueArray[head];
        }
    }
}
```

```

}
public int tail() {
    if(isEmpty()) {
        System.out.println("Queue is empty");
        return -1;
    }
    else {
        return queueArray[tail];
    }
}
}
}

```

05*Queue LinkedList:

Code for the Queue ADT using Nodes

We will use the same classes NodeData and Node used in the implementation of the LinkedList class and the Stack class

The class Queue has two data fields head and tail , both are references to Node

isEmpty() : Checks if the queue has no elements

enqueue(NodeData item) : Adds a new elements to the end of the queue

dequeue() : Removes and returns the front element of the queue

head() : returns the data of the first node in the queue

tail() : returns the data of the last node in the queue

```

class Queue {
    private Node head,tail;
    public Queue() {
        head = tail = null ;
    }
    public boolean isEmpty() {
        return head == null ;
    }
    public void enqueue(NodeData item) {
        Node newNode = new Node(item) ;
        if(isEmpty()) {
            head = tail = newNode ;
        }
        else {
            tail.next = newNode;
            tail = newNode;
        }
    }
    public NodeData dequeue() {
        if(!isEmpty()) {
            System.out.println("Queue is empty");
            return null;
        } else {
            NodeData value = head.data;
            head = head.next;
            if(head == null) {
                tail = null;
            }
            return value;
        }
    }
    public NodeData head() {
        if(isEmpty()) {
            System.out.println("Queue is empty");
            return null;
        } else {

```

```

return head.data;
}
}
public NodeData tail() {
    if(isEmpty()) {
        System.out.println("Queue is empty");
        return null;
    } else {
        return tail.data;
    }
}
}
class Nodes {
    NodeData data;
    Node next;
    public Node(NodeData data) {
        this.data = data;
        this.next = null;
    }
}
}

```

Chapter 16 : Algorithms Analysis

01*What is an Algorithm:

An algorithm is a finite , step-by-step procedure to solve a specific problem . It takes input , processes it , and produces an output. Algorithms must be

Finite: Ends after a number of steps

Definite: Each step is clearly defined

Effective: Each step is doable

Example: A simple algorithm to find the maximum number in an array

02*Algorithm Analysis:

Algorithm analysis is the process of evaluating the efficiency of an algorithm, mainly in terms of:

Time complexity(how fast it runs)

Space complexity (how much memory it uses)

03*Running Time:

Running time means the actual time an algorithm takes to execute based on:

Input size

Type of input

Computer speed

However, instead of measuring in seconds , we analyze it using primitive operations (assignments , comparisons , loops , etc)

04*Experimental Study:

This involves running the program with different inputs and measuring execution time using code (like in Java)

But it's limited because:

Results vary on different machines

Doesn't predict performance for very large inputs

05*Measuring Running Time Experimentally in Java:

You can measure execution time in Java using:

```
long start = System.nanoTime();  
//your code here  
long end = System.nanoTime();  
System.out.println("Running Time" + (end-start) + "ns");
```

This gives time in nanoseconds

06*Challenges of Experimental Analysis:

Hardware dependent.

Input dependent : Different inputs may result in different running times

Limited scalability : Can't easily test with huge inputs

07*Moving Beyond Experimental Analysis:

We use theoretical models:

Count primitive operations (basic steps like + , - , = , < , etc.)

Relate those to input size n

Use functions to express how the number of steps grows with n

08*Counting Primitive Operations:

Primitive operations include:

Assignments

Comparisons

Arithmetic operations

Function calls

Array indexing

Example:

```
int sum =0;  
for(int i=0;i<n ;i++)  
    sum+=i;
```

This runs n times , each doing 1 addition and 1 assignment -> Total : 2n primitive ops

09*Best , Worth , and Average Case:

Best Case : Minimum steps

Worst Case: Maximum steps

Average Case : Expected number of steps across all inputs

Example - Linear Search

Best Case: item in first => $O(1)$

Worst Case: item in last or missing => $O(n)$

Average Case => $O(n)$

10*The Seven Functions Used in the Analysis of Algorithms:

These functions describe growth rates - how fast the number of steps increases with n

1.Constant Function - $O(1)$:

Always takes the same time , regardless of input.

Example:

```
int x = 5;
```

Time doesn't grow with n

2. Logarithmic Function - $O(\log n)$:

Input size gets divided each time

Example:

```
while(n>1)
    n = n/2;
```

Used in Binary Search

3. Linear Function - $O(n)$:

Steps grow directly with input size

Example:

```
for(int i =0;i<n;i++)
    sum+=arr[i];
```

4. Linear Arithmetic Function - $O(n \log n)$:

Appears in efficient sorting algorithms like Merge Sort and Quick Sort (on average).

5. Quadratic Function - $O(n^2)$:

Used in algorithms with nested loops over the input

Example:

```
for(int i=0;i<n;i++)
    for(int j=0;j<n;j++)
        sum +=(i+j);
```

Common in simple sorting like Bubble Sort

6. Cubic Function - $O(n^3)$:

Three nested loops

Example:

```
for(int i=0;i<n;i++)
    for(int j=0;j<n;j++)
        for(int k=0; k<n ;k++)
            doSomething();
```

7. Exponential Function - $O(2^n)$:

Time doubles with each increase in n .

Example:

Recursive Fibonacci

```
int fib(int n) {
    if(n<=1)
        return n;
    return fib(n-1) + fib(n-2) ;
}
```

Extremely slow for large n

11* Comparing Growth Rates:

From fastest to slowest:

$O(1) < O(\log n) < O(n) < O(n \log n) < O(n^2) < O(n^3) < O(2^n)$

Lower growth = better scalability

12* Comparative Analysis:

Helps choose the best algorithm by comparing their time complexities

Example:

Linear Search: $O(n)$

Binary Search : $O(\log n)$

Use Binary Search for sorted data because it's faster

13*Examples of Algorithm Analysis:

1.Constants & Loops:

```
for(int i=0 ; i<n ;i++)  
    for(int j=0; j<5 ; j++)  
        //do something
```

This is $O(n)$ because the inner loop is constant (5)

2.Finding the Maximum in an Array:

```
int max = arr[0];  
for(int i = 1 ; i< arr.length ;i++) {  
    if(arr[i]>max)  
        max = arr[i];  
}
```

Time complexity : $O(n)$ - goes through each element once

Chapter 17: Recursion

01*What is Recursion?

Recursion is a programming technique where a method calls itself to solve a problem by breaking it into smaller , simpler instances of the same problem

02*Components of a Recursive method:

Every recursive method must have:

Base Case:A condition to stop the recursion . Without it , the method would call itself infinitely

Recursive Call: A call the same method with a smaller or simpler input that moves the problem closer to the base case

03*Example - Factorial using Recursion (returns int):

```
public static int factorial(int n) {  
    if(n==0 || n==1)  
        return 1;  
    return n*factoriel(n-1);  
}
```

Base Case: if $n==0$ or $n==1$, return 1.

Recursive Case: Multiply n by the result of $\text{factorial}(n-1)$

04*Void vs Non-Void Recursive Methods:

A.Non-Void Recursion (returns a value):

These methods return a result. For example:

Here, the method returns the final result after all recursive calls

```
public static int sum(int n) {  
    if(n==1)  
        return 1;  
    return n + sum(n-1);  
}
```

B.Void Recursion(does not return a value):

These methods perform actions like printing or modifying data , without returning anything

```
public static void countdown(int n) {  
    if(n==0)  
        return;  
    System.out.println(n);  
    countdown(n-1);  
}
```

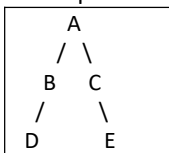
Chapter 18: Tree

01*Trees Definitions:

1.Tree:

A non-linear data structure made of nodes

Example:



2.Root:

The topmost node of the tree

Example:

in the above tree , node A is the root

3.Leaf:

A node that has no children

Example:

Nodes C , D and E are leaves

4.Branch:

A path from the root to any leaf

Example:

One branch is A -> B -> D

5.Weight:

The total number of nodes in a tree or subtree

Example:

Full tree: 5 Nodes(A,B,C,D,E) -> weight= 5

Subtree starting at B : nodes B, D, E -> weight = 3

6.Tree:

Distance from the root node

Example:

A is at level 0

B and C are at level 1

D and E are at level 2

7.Height:

The longest path from the root to a leaf (in number of edges)

Example:

A -> B -> D = 2 edges -> height = 2

8.Parent:

A node that has child nodes

Example:

A is parent of B and C
B is parent of D and E

9.Child:

A node that comes from a parent node

Example:

B is child of A
E is child of B

10.Sibling:

Nodes that share the same parent

Example:

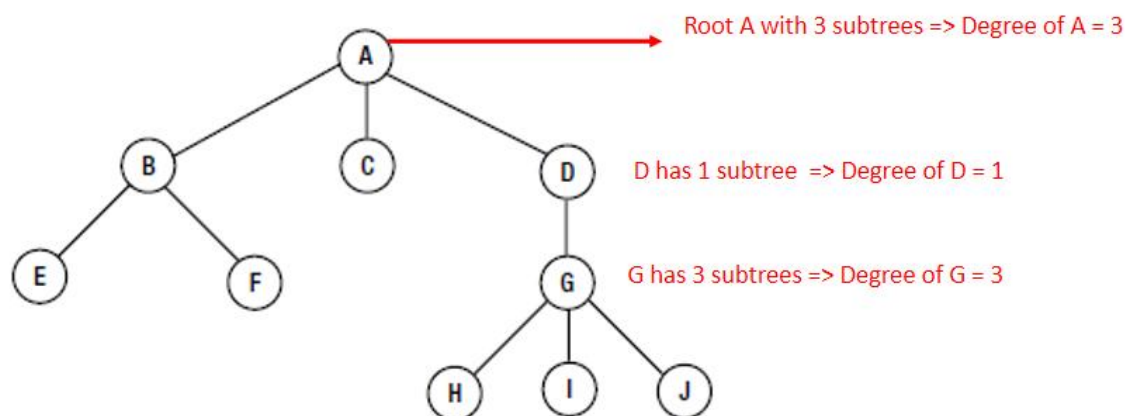
D and E are siblings (they have same parent B)
B and C are siblings (parent is A)

11.Degree:

The degree of a node is the number of subtrees of the node. Think of it as the number of lines leaving the node.

For example

degree(A) = 3 , degree(B) = 2 , degree(C) = 0 , degree(D) = 1 , and degree(G) = 3



02*Binary Trees:

1.Binary Trees:

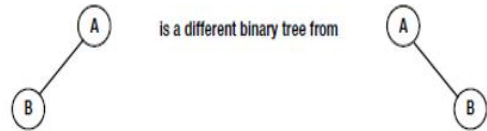
In a binary tree, a node may have 0, 1 or 2 children maximum

Example 1:

Here's a binary tree with one node, the root:

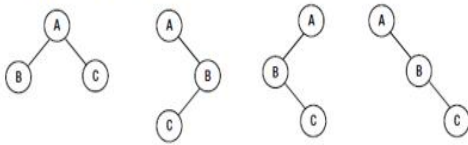


Example 2:



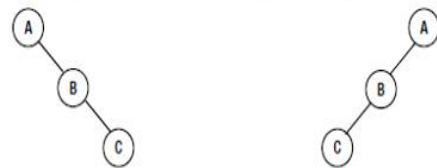
Example 3:

Here are binary trees with three nodes:

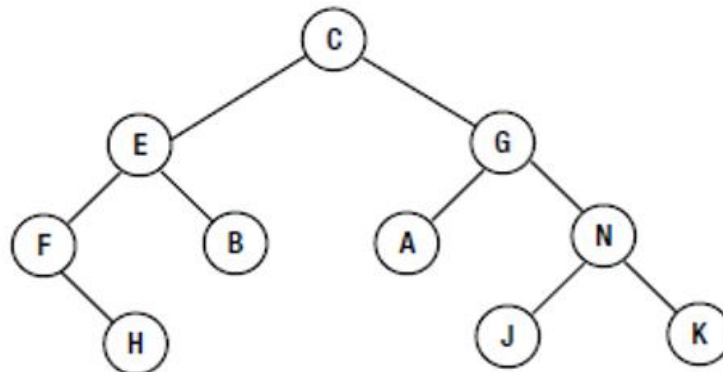


Example 4:

Here are binary trees with all left subtrees empty and all right subtrees empty:

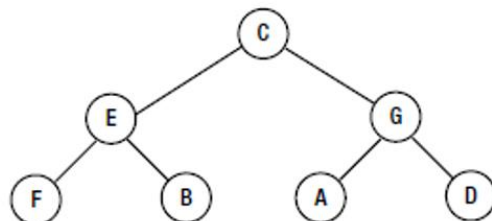


Example 5: General Binary Tree



Example 5: A complete binary tree

A complete binary tree is a binary tree where each node, except the leaves, has exactly two subtrees.

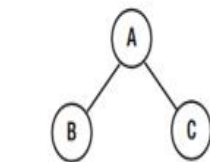


2.Traversing a Binary Tree:

Pre-order:

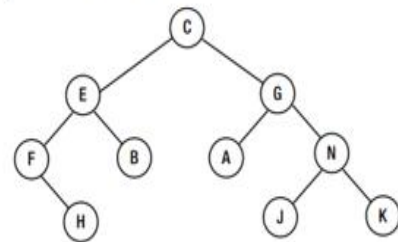
- 1.Visit the root
- 2.Traverse the left subtree in pre-order
- 3.Traverse the right subtree in pre-order

The *pre-order* traversal of this tree



is A B C.

The *pre-order* traversal of this tree



is C E F H B G A N J K.

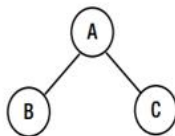
preOrder Traversal -> Node Left Right (NLR):

```
public void preOrder() {  
    preOrderTraversal(root);  
}  
private void preOrder(TreeNode current) {  
    if(current!=null) {  
        System.out.print(current.data + ' ');  
        preOrderTraversal(current.left);  
        preOrderTraversal(current.right);  
    }  
}
```

Inorder:

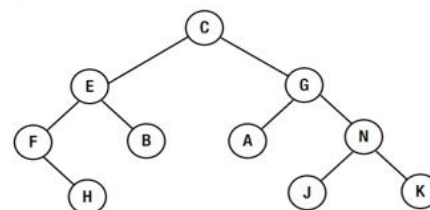
- 1.Traverse the left subtree in in-order
- 2.Visit the root
- 3.Traverse the right subtree in in-order

The *in-order* traversal of this tree



is B A C.

The *in-order* traversal of this tree



is F H E B C A G J N K

```

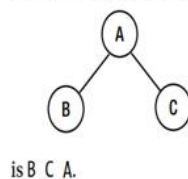
inOrder Traversal -> Left Node Right (LNR)
public void inOrder() {
    inOrderTraversal(root);
}
private void inOrderTraversal(TreeNode current) {
    if(current!=null) {
        inOrderTraversal(current.left);
        System.out.print(current.data + " ");
        inOrderTraversal(current.right);
    }
} //end inOrderTraversal

```

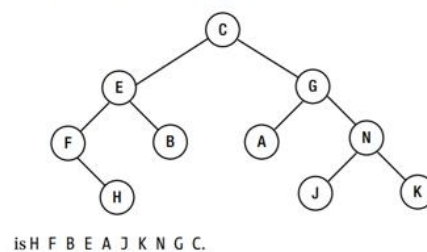
Post-order:

- 1.Traverse the left subtree in post-order
- 2.Traverse the right subtree in post-order
- 3.Visit the root

The *post-order* traversal of this tree



The *post-order* traversal of this tree



postOrder Traversal -> Left Right Node (LRN)

```

public void postOrder() {
    postOrderTraversal(root);
}
private void postOrderTraversal(TreeNode current) {
    if(current !=null) {
        postOrderTraversal(current.left);
        postOrderTraversal(current.right);
        System.out.print(current.data + " ");
    }
}

```

How to represent a Binary Tree:

Each node of a binary tree consists of three fields:

- .A field containing the data at the node
- .A reference to the left subtree
- .A reference to the right subtree

We can begin by writing the class `TreeNode` which is abit similar to the class `Node` used with the `LinkedList` class

```

class TreeNode {
    NodeData data;
    TreeNode left, right;
    public TreeNode(NodeData d) {
        data = d;
        left = right = null;
    }
}
class NodeData {
    int value;
    public NodeData(int v) {
        value = v;
    }
}

```

```

}
class BinaryTree {
    private TreeNode root;
    public BinaryTree() {
        root = null;
    }
    //other methods
}

```

03*Binary Search Trees (BST):

1.A Binary Search Tree (BST) is a binary tree which has the following properties:

The left sub-tree of a node has a key less than to its parent node's key

The right sub-tree of a node has a key greater than to its parent node's key

In other words:

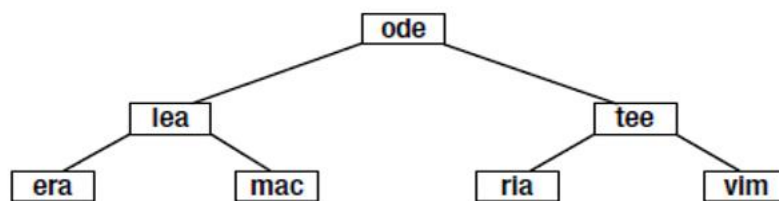
For every node x in the tree , if y is in that node's left subtree , then $y < x$

For every node x in the tree, if y is in that node's right subtree , then $y > x$

2.BST Example:

Given any node , a word in the left subtree is "smaller" and a word in the right subtree is "greater" than the word at the node . (Here , smaller and greater refer to alphabetical order based on the first letter.)

It facilitates the search for a given key using a method of searching similar to the binary search of an array



Code for the search operation:

```

public boolean search(NodeData item) {
    TreeNode current = root;
    if(isEmpty())
        System.out.print("The search operation failed. The BST is empty");
    else
        while(current!=null) {
            if(item.compareTo(current.data) == 0)
                return true;
            else if(item.compareTo(current.data) < 0)
                current = current.left;
            else
                current = current.right;
        }
    return false;
}

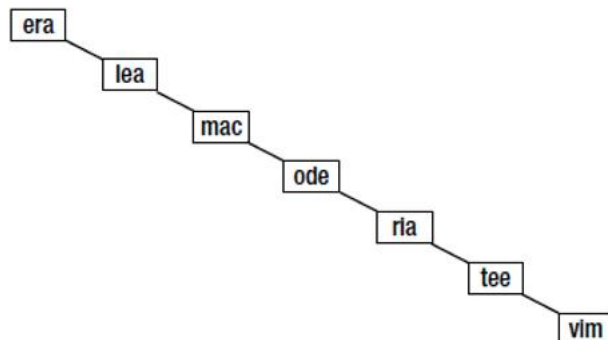
```

The worst case(Degenerate Tree):

If the words come in alphabetical order , then the tree build will be like this

Searching such a tree is reduced to a sequential search of a linked list

This kind of tree is called a degenerate tree



Inserting an element into a BST:

In the previous slides we covered the method search used to search for an item in a binary search tree (BST)

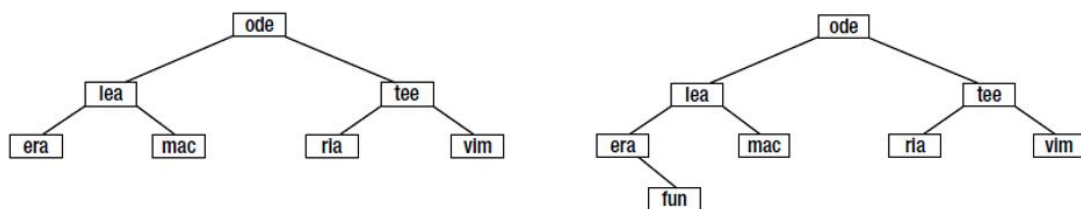
In the next slides we look at the method findOrInsert used to insert an item a binary search tree (BST) if it is not already found there

Adding the Non Existing word "fun":

When explaining the search algorithm , we discovered that if fun is to exist in the BST it must be on the right of era

Since it is not there, if we need to add it, then it must be added to the right of era

Not only does the binary search tree facilitates searching, but if an item is not found , it can easily inserted



```
public void findOrInsert(NodeData item) {
    TreeNode current = root, parent = null, newNode;
    boolean found = false;
    if(isEmpty())
        root = new TreeNode(item);
    else {
        while(current!=null) {
            if(item.compareTo(current.data) == 0) {
                System.out.println("Insert Operation failed.The element is already in the BST");
                found =true;
                break;
            }
            else if(item.compareTo(current.data) < 0) {
                parent = current;
                current = current.left;
            }
            else {
                parent = current;
            }
        }
    }
}
```

```

current = current.right;
}
}
if(!found) {
    newNode = new TreeNode(item);
    if(item.compareTo(parent.data) < 0)
        parent.left = newNode;
    else
        parent.right = newNode;
}
}
}

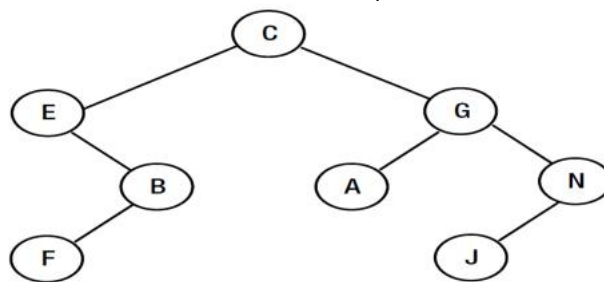
```

04*Level-Order Traversal:

An alternative to pre-order, in-order, and post-order traversals is level-order

We traverse the tree level by level (from left to right in each level)

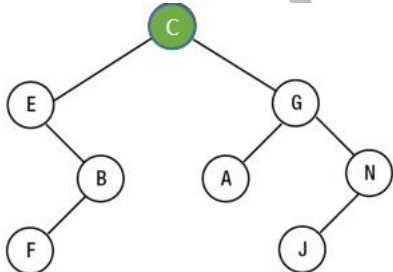
To perform a level-order traversal, we use a queue of `TreeNode` objects



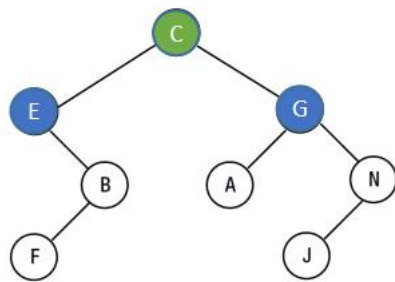
Its level-order traversal is C E G B A N F J.

Level-Order Traversal using a queue

Enqueue C in Q

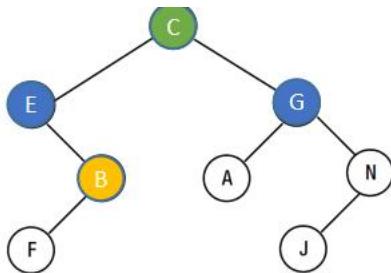


Q is not empty, so dequeue and visit (output) C, then add (enqueue) its children E and G to Q



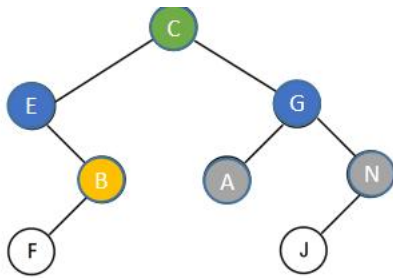
Output: C

Q is not empty , so dequeue and visit (output) E, then add its child B to Q



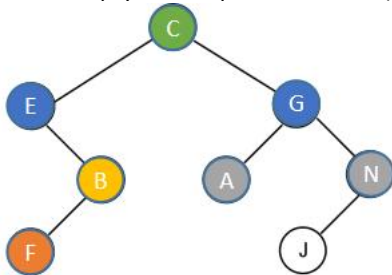
Output: C E

Q is not empty , so dequeue and visit (output) G , then add its children A and N to Q



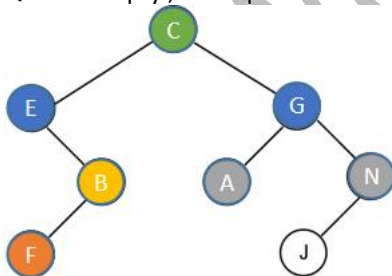
Output: C E G

Q is not empty , so dequeue and visit (output) B , then add its child F to Q



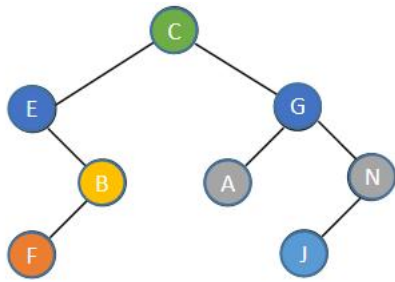
Output: C E G B

Q is not empty , so dequeue and visit (output) A . Since A has no children then nothing is added to Q



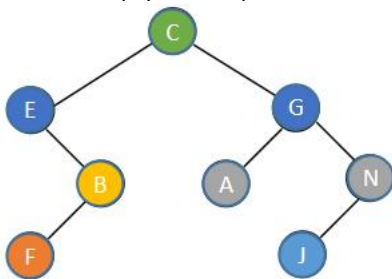
Output: C E G B A

Q is not empty , so dequeue and visit (output) N , then add its child J to Q



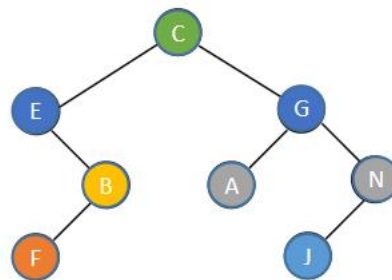
Output: C E G B A N

Q is not empty , so dequeue and visit (output) F . Since F has no children then nothing is added to Q



Output: C E G B A N F

Q is not empty , so dequeue and visit (output) J . Since J has no children then nothing is added to Q



Output: C E G B A N F J

Now Q is empty . The traversal ends having visited the nodes in the order C E G B A N F J

05*Some useful tree functions:

Count the number of nodes in a tree:

```
public int numNodes() {  
    return countNodes(root);  
}  
private int countNodes(TreeNode current) {  
    if(current == null)  
        return 0;  
    return 1 + countNodes(current.left) + countNodes(current.right);  
}
```

Note: the public method is not the recursive one

Count the number of leaves in the tree . That is the number of nodes that have their left and right nodes both null.

Some Useful Binary Tree Functions:

```
public int numLeaves() {  
    return countLeaves(root);  
}  
private int countLeaves(TreeNode current) {  
    if(current == null)  
        return 0;  
    if(current.left == null && current.right == null)  
        return 1;  
    return countLeaves(current.left) + countLeaves(current.right);  
}
```

Note: Same recursion principle as the countNode method

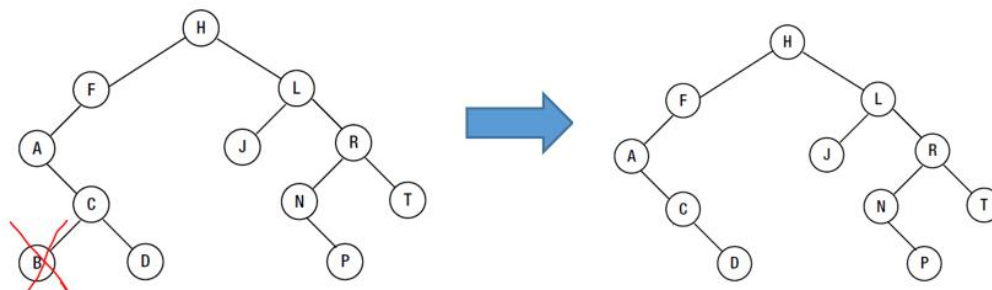
Calculate the height (number of levels) of the tree:

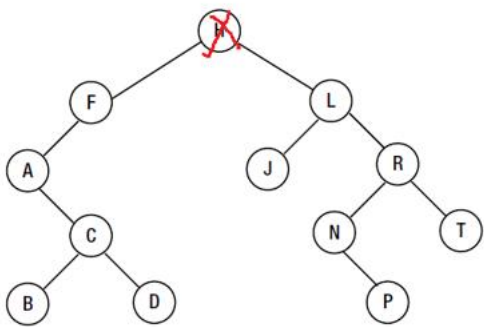
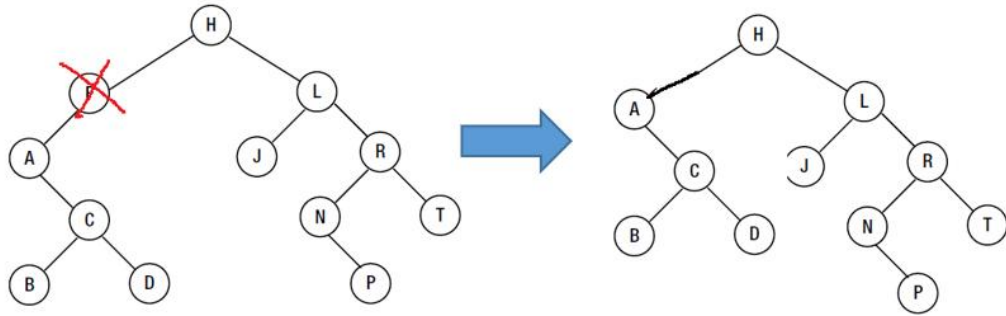
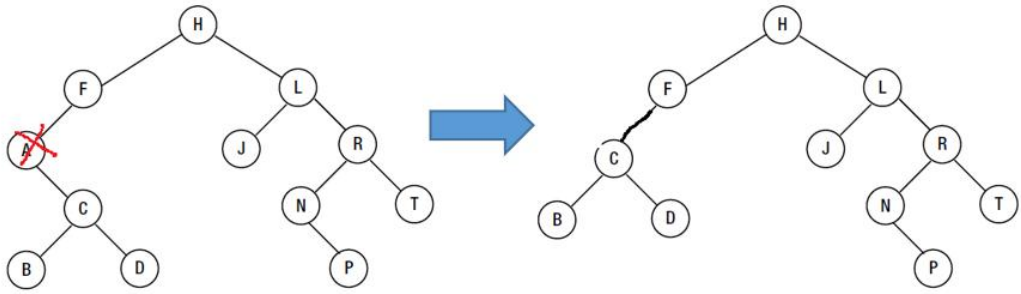
```
public int height() {  
    return countLevels(root);  
}  
private int countLevels(TreeNode current) {  
    if(current == null)  
        return 0;  
    return 1 + Math.max(countLevels(current.left), countLevels(current.right));  
}
```

06*BST deletion:

1.replace H with J called in-order successor then delete the successor

2.replace H with F called in-order predecessor then delete the predecessor





Code for method delete:

```
public void delete(NodeData item) {
    TreeNode parent = null , child , temp , current = root ;
    boolean found = false;
    if(root == null) {
        System.out.println("Delete operation failed. The BST is empty.");
        return ;
    }
    while(current != null && !found) {
        if(item.compareTo(current.data) == 0)
            found = true;
        else if(item.compareTo(current.data) < 0) {
            parent = current;
            current = current.left;
        }
        else {
            parent = current ;
            current = current.right;
        }
    }
    if(!found)
        System.out.println("Delete failed. Element is not in the BST");
    else {
        if(current.left != null && current.right != null) {
            temp = current;
            current = current.left;
        }
        temp.data = current.data;
    }
    child = current.left;
    if(child == null)
        child = current.right;
    if(current == root)
        root = child;
    else if(parent.left == current)
        parent.left = child;
    else
        parent.right = child;
}
```

Chapter 19: Graphs

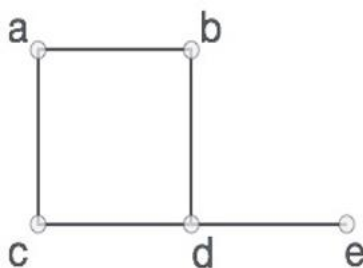
01*Graph Terminology:

A graph is a representation of a set of objects where some pairs of objects are connected by links . The interconnected objects are represented by points termed as vertices (nodes) , and the links that connect the vertices are called edges

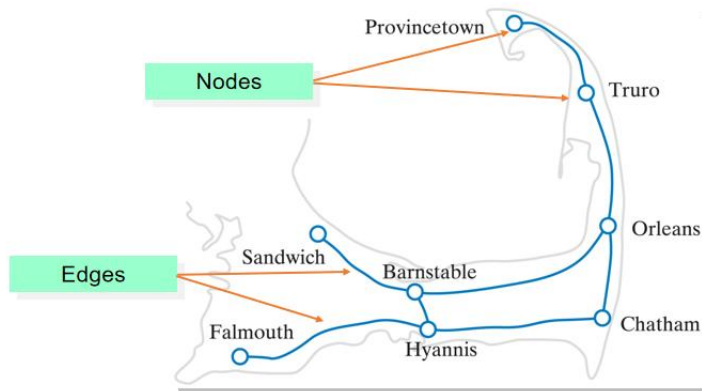
Formally, a graph is a pair of sets (V,E), where V is the set of vertices and E is the set of edges, connecting the pairs of vertices. Take a look at the following graph

$V = \{a, b, c, d, e\}$

$E = \{ab, ac, bd, cd, de\}$



Graph Example(Road Map):



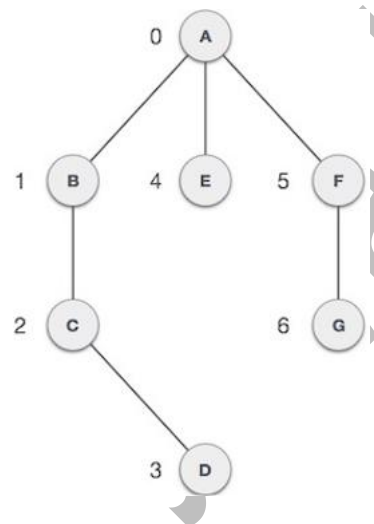
02*Graph Data Structure:

Vertex - Each node of the graph is represented as a vertex. In the following example, the labeled circle represents vertices

Edge - Edge represents a path between two vertices or a line between two vertices. In the following example, the lines from A to B , B to C , and so on represents edges

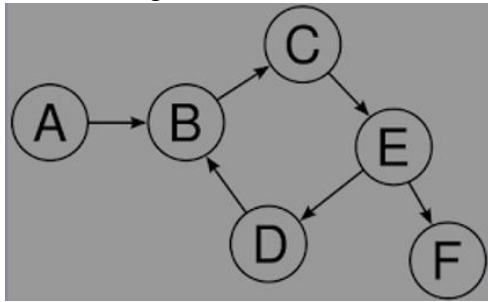
Adjacency - Two nodes or vertices are adjacent if they are connected to each other through an edge . In the following example, B is adjacent to A , C is adjacent to B , and so on.

Path - Represents a sequence of edges between the two vertices . In the following example , ABCD represents a path from A to D



03*Directed Graphs:

Each edge of a directed graph or digraph has a specific direction denoted by arrows. A directed edge is also called unidirectional a directed graph



Directed Graphs Examples:

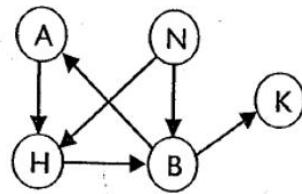


Figure 5.2: A directed graph with 5 nodes and 6 edges

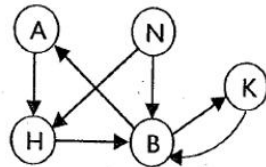
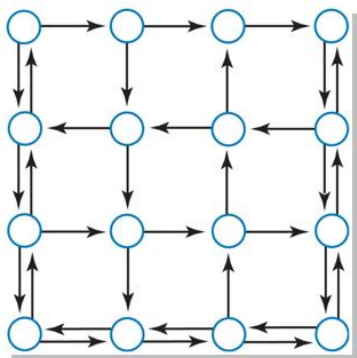
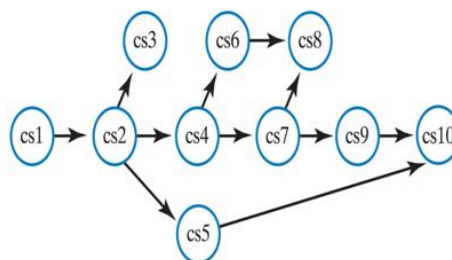


Figure 5.3: Graph with 2 edges connecting the same nodes

Road Maps



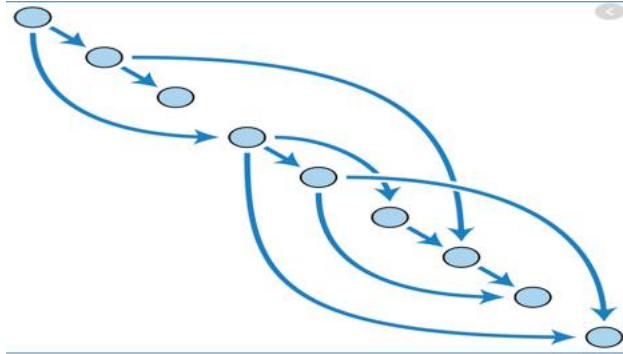
Course Prerequisites



04*Directed Acyclic Graph (DAG):

A path that starts and ends at the same vertex is called a cycle

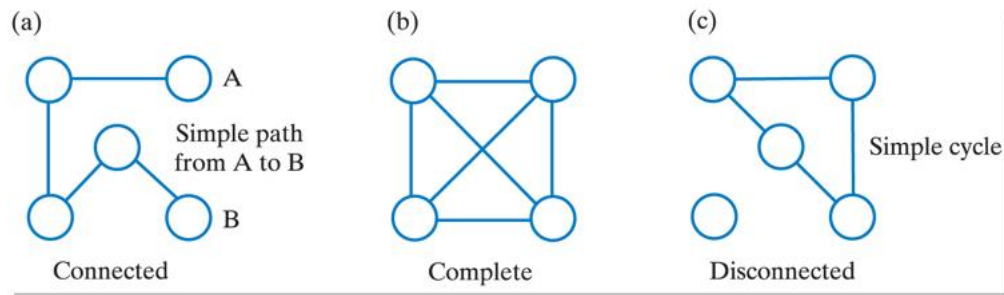
A directed acyclic graph is a directed graph with no cycles



05*Undirected Graphs:

Edges in an undirected graph have no directions

An undirected edge is also called bidirectional



06*Weighted Graphs:

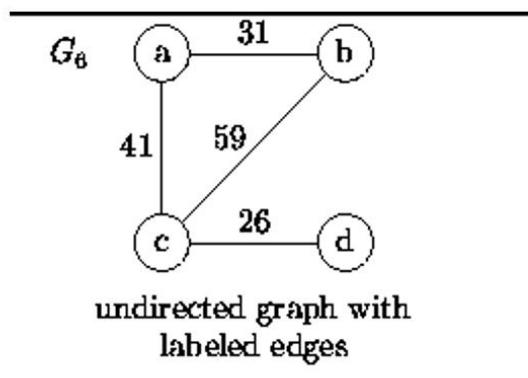
A weighted graph has values (weight / costs) associated with its edges

A path in a weighted graph also has weight which is the sum of the sum of the edge weights

Examples of weights:

- Miles between nodes on a map
- Driving time between nodes
- Taxi cost between node locations

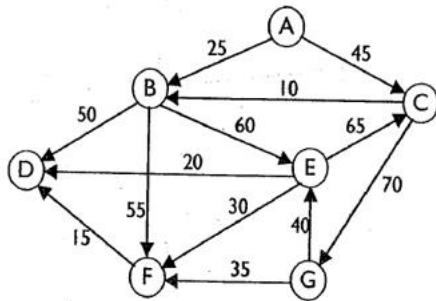
Weighted Graph Example:



07*Graph Representation:

There are two ways to represent a graph:

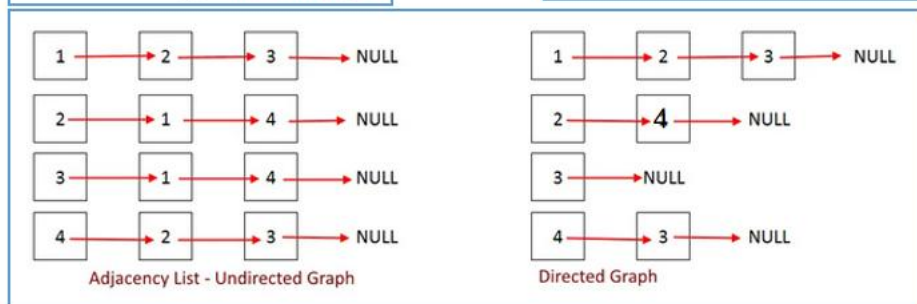
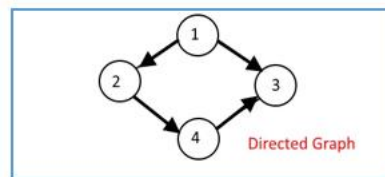
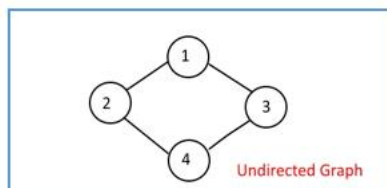
1.Adjacent Matric:



	1	2	3	4	5	6	7
1	0	25	45	∞	∞	∞	∞
2	∞	0	∞	50	60	55	∞
3	∞	10	0	∞	∞	∞	70
4	∞	∞	∞	0	∞	∞	∞
5	∞	∞	65	20	0	30	∞
6	∞	∞	∞	15	∞	0	∞
7	∞	∞	∞	∞	40	35	0

Adjacency matrix, G

2.Adjacency List:



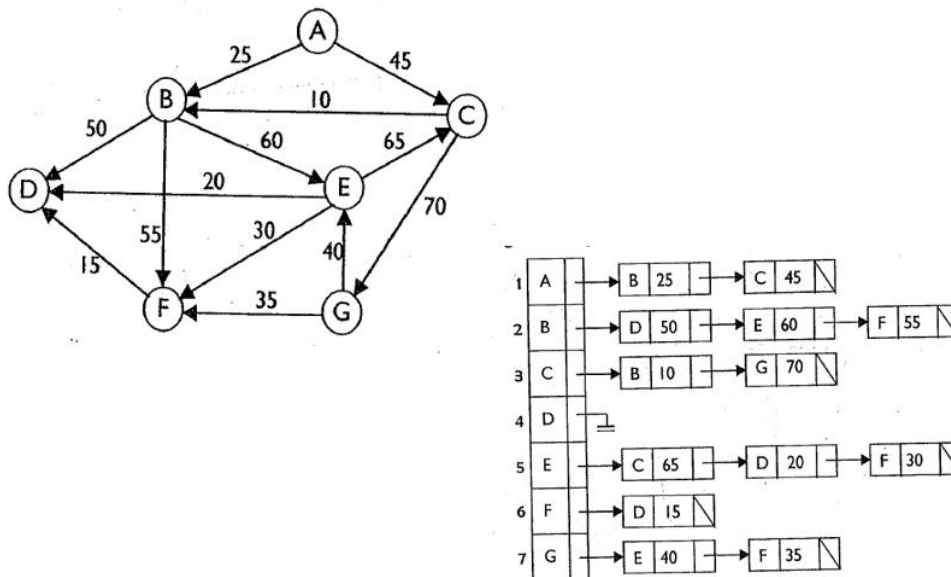
3.Adjacency matrix:

The adjacency matrix is a 2 dimensional array of size $V \times V$, where V is the number of vertices in the graph. It represents the adjacent vertices

4.Adjacency List:

The adjacency List is a one dimensional array of linked lists, where the array size is the same as number of vertices in the graph. Every Vertex in the graph has a LinkedList. Each Node in the LinkedList represents the reference to the other vertices which share an edge with the current vertex. The weights can also be stored in the LinkedList Node

Example of adjacency list:



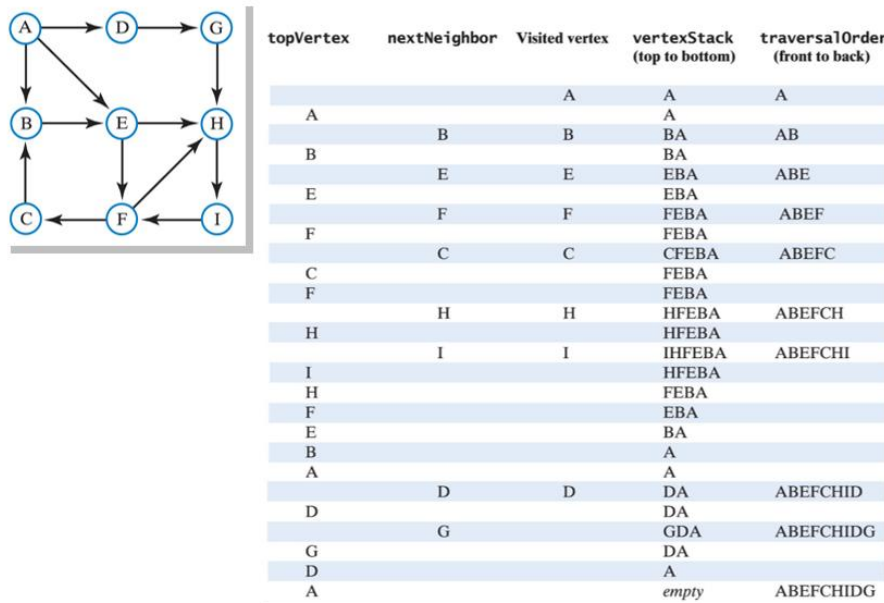
08*Depth-First Traversal:

Visits a vertex, then a neighbor of the vertex, then a neighbor of the neighbor, etc., advance as much as possible from the original vertex then back up by one vertex. Consider the next neighbor.

The DFS algorithm works as follows:

1. Start by putting any one of the graph's vertices on top of a stack
2. Take the top item of the stack and add it to the visited list
3. Create a list of that vertex's adjacent nodes. Add the ones which aren't in the visited list to the top of the stack
4. Keep repeating steps 2 and 3 until the stack is empty

Depth-First Traversal Example with the stack:



We now look at the other common way of traversing a graph - Breadth-first Traversal.

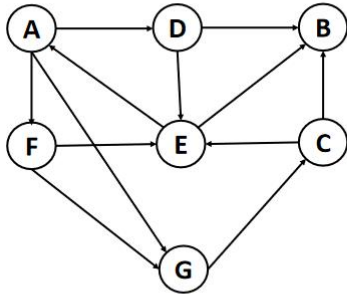
A breadth-first traversal

visits a vertex and then each of the vertex's neighbors before advancing

This is similar to level-order traversal in a binary tree

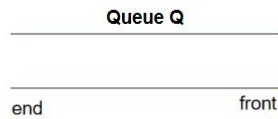
09*Breadth-first Traversal:

We now look at the other common way of traversing a graph - Breath-first Traversal

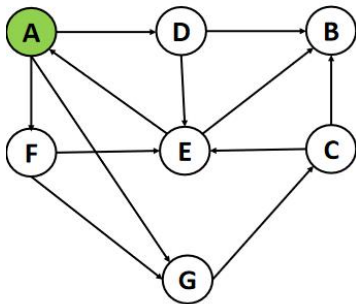


Steps:

Let us look at the details of how a breadth-first search works.

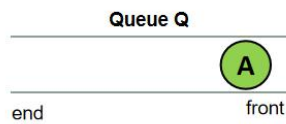


Output:

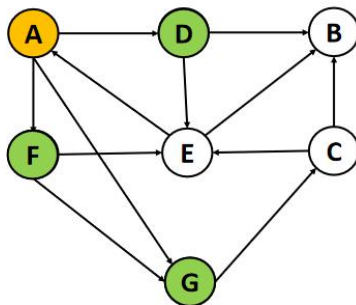


Steps:

Starting from A. Mark and enqueue A.

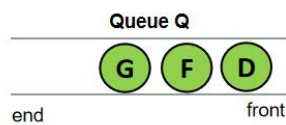


Output:

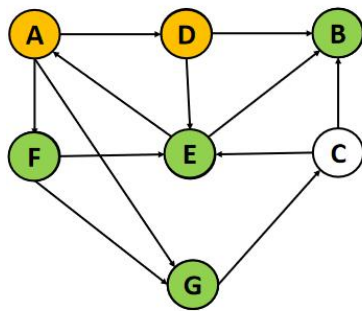


Steps:

Dequeue and Visit A. A is connected to D, F and G via (A,D), (A,F) and (A,G) edges respectively. So mark and enqueue D, F and G.

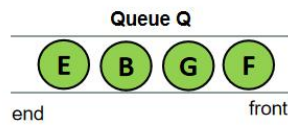


Output: A

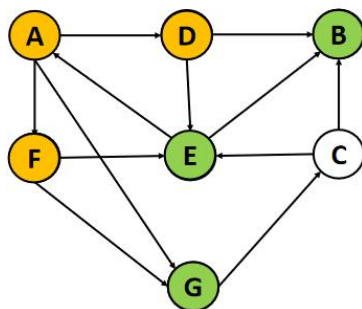


Steps:

Dequeue and Visit D. D is connected to B and E via (D,B), and (D,E) edges respectively.
So mark and enqueue B and E.

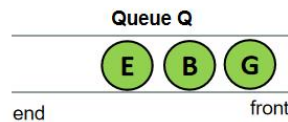


Output: A D

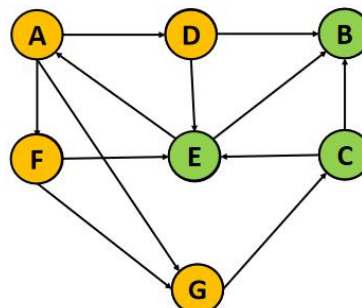


Steps:

Dequeue and Visit F. F is connected to E and G via (F,E) and (F,G) edges respectively. There is no thing to do since E and G have already been seen and equeued to Q.

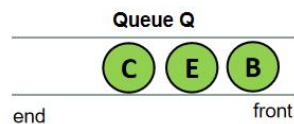


Output: A D F

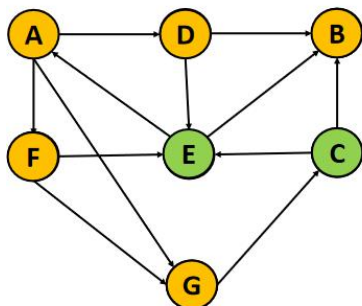


Steps:

Dequeue and Visit G. G is connected to C via (G,C) edge.
Mark and equeue C.

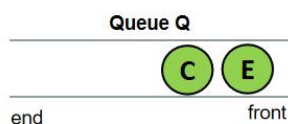


Output: A D F G

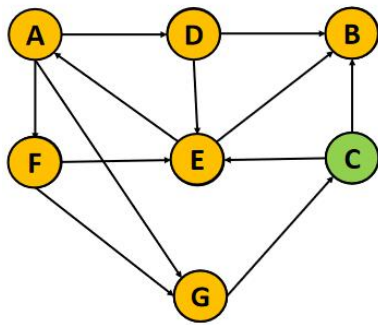


Steps:

Dequeue and Visit B. There are no edges leaving B, so there is nothing to do.

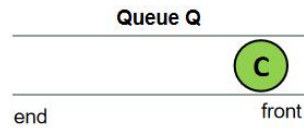


Output: A D F G B

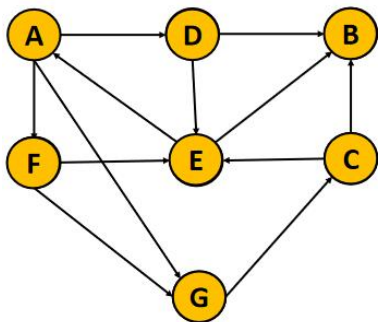


Steps:

Dequeue and Visit E. E is connected to A and B via (E, A) and (E, B) edges respectively. There is no thing to do since A and B have already been visited.

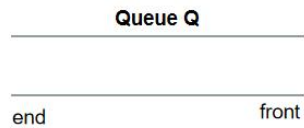


Output: A D F G B E

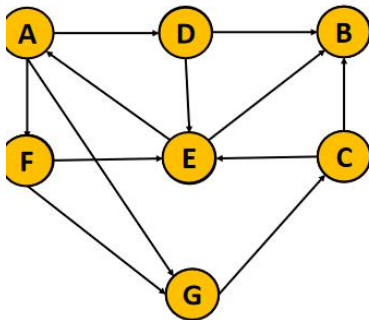


Steps:

Dequeue and Visit C. C is connected to B and E via (C, B) and (C, E) edges respectively. Also there is no thing to do since B and E have already been visited.

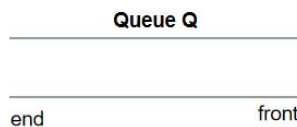


Output: A D F G B E C



Steps:

Q is empty so the algorithm terminates.



Output: A D F G B E C

12*Dijkstra Algorithm:

Dijkstra's algorithm is an algorithm for finding the shortest paths from a source node S to all other nodes in a graph

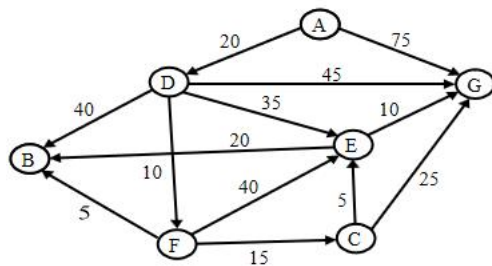
Shortest Path:

Suppose we want to find the shortest path from A to E

A → D → E (with a cost of 55)

A → D → F → E (cost 70)

A → D → F → C → E (cost 50)



Important observation : while we may be interested in finding the shortest path from one vertex to another specific vertex , it turns out that it has the same complexity of finding the shortest path from the source vertices

How Dijkstra works:

We first assume that the weights of edges are non-negative

We want to find the shortest path from s to all other vertices in the graph

We grow a “cloud” of vertices, beginning with s and eventually covering all the vertices

The distance of a vertex v a label $d(v)$ is the length of a shortest path between s and v

We store with each vertex v a label $d(v)$ representing the distance of v from s in the subgraph consisting of the cloud and its adjacent vertices

At each step

We add to the cloud the vertex u outside the cloud with the smallest distance label, $d(u)$

We update the labels of the vertices adjacent to u

13*Algorithm:

1)Create a set Marked that keeps track of already visited vertices. Initially , this set is empty

2)Assign a distance value to all vertices in the graph . Initialize all distance value as INFINITE. Assign distance value as 0 for the source so that it is picked first

3)While Marked doesn't include all vertices

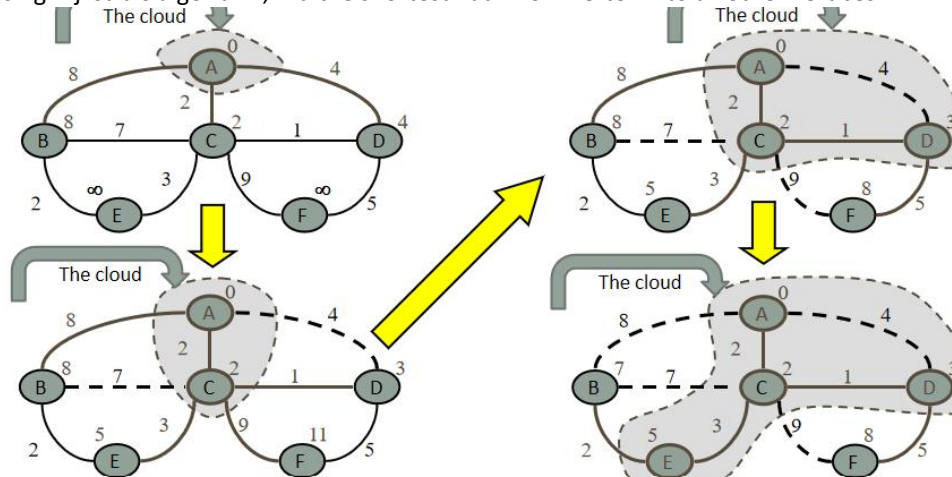
a-pick a vertex u which is not there in Marked and has a minimum distance value

b-Include u to Marked

c-Update distance value of all adjacent vertices of u. To update the distance values . iterate through all adjacent vertices. For every adjacent vertex v , if the sum of distance value of u (from source) and weight of edge u-v , is less than the distance value of v , then update the distance value of v

Example:

Using Dijkstra's algorithm, find the shortest Path from vertex A to all other vertices



Shortest Path from A to all other vertices:

A to B : A -> C -> E -> B with cost = 7

A to C: A -> C with cost = 2

A to D: A -> C -> D with cost = 3

A to E: A -> C -> E with cost = 5

A to F: A -> C -> D -> F with cost = 8

Chapter 20: HashTable & HashMap

01*What is a Hash Table?

A Hash Table is a data structure used to store data in the form of key-value pairs

Each key is associated with exactly one value

The main purpose of a hash table is to provide very fast access to data

Example idea(conceptual):

Key: "Jawad" -> Value: 1200

Key: "Ali" -> Value: 1000

02*Why Hash Tables Are Fast?

-Hash tables do not search data sequentially like arrays of lists

-Instead, they use a hash function to calculate a position(index) where the data should be stored

-In most cases, the time complexity is $O(1)$

-Because of this: Insertion is fast, Searching is fast, Deletion is fast

03*Hash Function:

-A hash function converts a key into an integer value called a hash code

-The hash code is then converted into an array index

General idea:

$\text{index} = \text{hashCode}(\text{key}) \% \text{arraySize}$

04*Collision:

-A collision happens when two different keys produce the same index

-This is normal and expected in hash tables

-Java handles collisions using chaining:

Each index in the internal array can store multiple key-value pairs

These pairs are stored in a linked structure

05*HashMap vs Hashtable:

HashTable is the old version : Thread-safe / slower / Does not allow null keys or null values

HashMap is the modern version : Not thread-safe by default / Faster / Allows one null key and multiple null values

In modern Java , HashMap is preferred

06*HashMap:

HashMap is a class from the java.util package

Declaration:

```
HashMap<KeyType, ValueType> map = new HashMap<>();
```

Example:

```
HashMap<Integer , String> students = new HashMap<>();
```

07*Inserting Data into HashMap:

Using put() :

Each key must be unique

If you insert a value with an existing key , the old value is replaced

```
students.put(1 , "Jawad");
```

```
students.put(2 , "Ali");
```

```
students.put(3 , "Hassan");
```

08*Retrieving Data from HashMap:

Using get():

```
System.out.println(students.get(2));
```

Output

Ali

If the key doesn't exist , get() return null

09*Checking Keys and Values:

To check if a key exists:

```
students.containsKey(1);
```

To check if a value exists:

```
students.containsValue("Jawad");
```

Both return either true or false;

10*Removing Data:

```
Student.remove(2);
```

```
System.out.println(students);
```

Output:

```
{1=Jawad, 3=Hassan}
```

11*Iterating Over HashMap:

One way is by using the keys:

```
for(Integer key: student.keySet()) {  
    System.out.println(key + "->" + students.get(key));  
}
```

12*Null in HashMap:

HashMap allows:

One null key

Multiple null values

Example:

```
HashMap<String, Integer> map = new HashMap<>();  
map.put(null, 5);  
map.put("A", null);  
System.out.println(map);
```

Output

```
{null = 5, A=null}
```

13*Internal Structure:

-Internally, HashMap uses:

An array of buckets

Each bucket stores entries

Uses hashing to decide the bucket

14*When to Use HashMap:

-Use HashMap when:

Data is stored as key-value pairs

Fast access is required

Order of elements does not matter

-Do not use HashMap if:

You need sorted keys

You need insertion order

Chapter 21 : Vector

01*Introduction to Vector:

Vector is one of the data structures provided by Java. It is used to store multiple elements in a single object.

A Vector works like an array, but it has a dynamic size. This means that the Vector can automatically grow when more elements are added.

Vector is part of the Java Collections Framework and is found in the java.util package.

Example:

```
Vector<Integer> numbers = new Vector<>();  
numbers.add(10);  
numbers.add(20);  
numbers.add(30);  
System.out.println(numbers);
```

Output:

```
[10, 20, 30]
```

02*Definition of Vector:

A Vector is a dynamic array that stores elements in a continuous memory structure and automatically increases its capacity when the current storage is full.

Vector stores elements in an ordered way.

Each element has an index.

Index starts from 0.

Vector size can change during program execution.

Vector can store objects of a specific data type using Generics.

Accessing:

```
v.get(1);
```

Output:

```
10
```

03*Features of Vector:

1.Dynamic Size:

Unlike arrays, Vector size is not fixed.

Array:

```
int arr[] = new int[3];
```

The size is always: 3

Vector:

```
Vector<Integer> v = new Vector<>();
```

The size can become:

3 → 10 → 20 → ... depending on the number of elements.

2.Stores Multiple Elements:

A Vector can store many values of the same type.

Example:

```
Vector<String> names = new Vector<>();  
names.add("Ali");  
names.add("Ahmad");  
names.add("Sara");
```

Output:

```
[Ali, Ahmad, Sara]
```

3.Index-Based Access:

Each element has an index.

Example:

Index:	0	1	2
	Ali	Ahmad	Sara

Access:

```
names.get(2);
```

Output:

```
Sara
```

4.Thread Safety:

Vector methods are synchronized, which means they are designed to support multiple threads accessing the Vector safely.

However, this synchronization can make Vector slower compared with other dynamic arrays like ArrayList.

04*Why Use Vector?

We use Vector when we need a collection that can change its size during execution.

Problem with Normal Arrays

Arrays have a fixed size.

Example:

```
int numbers[] = new int[3];
```

Memory:

10 20 30

If we want to add another value:

40

there is no available space.

We must create a new larger array and copy all elements.

Solution:

Vector handles resizing automatically.

Before adding:

Capacity = 3

10 20 30

After adding a new element:

Capacity increases

10 20 30 40

05*Vector Structure:

A Vector internally uses an array to store its elements.

The structure of a Vector contains three important parts:

1.Internal Array:

Vector uses an internal array as the actual storage.

Example:

```
Vector<Integer> v = new Vector<>();  
v.add(10);  
v.add(20);  
v.add(30);
```

The empty spaces are available for future elements.

2.Size:

Size represents the number of elements currently stored in the Vector.

Example:

```
Vector<Integer> v = new Vector<>();  
v.add(5);  
v.add(10);  
v.add(15);
```

Size:

size = 3

```
System.out.println(v.size());
```

Output:

3

3.Capacity:

Capacity represents the total number of elements that the Vector can store before it needs to increase its size.

Example:

Capacity = 5

Size = 3

Here:

size = 3

capacity = 5

The Vector has two empty spaces available.

```
System.out.println(v.capacity());
```

4.Dynamic Resizing:

When the Vector becomes full, it automatically creates a larger array and copies the old elements.

Example:

Before:

Capacity = 3

1	2	3
---	---	---

Adding:

40

After resizing:

Capacity = 6

06*Creating a Vector:

Before using Vector, we need to import the Vector class.

```
import java.util.Vector;
```

1.Creating an Empty Vector:

Syntax:

```
Vector<DataType> name = new Vector<>();
```

Example:

```
Vector<Integer> numbers = new Vector<>();
```

Output:

[]

The Vector is created but contains no elements.

2.Creating Vector with Initial Capacity:

We can specify the initial capacity.

Example:

```
Vector<Integer> numbers = new Vector<>(10);
```

Capacity = 10

Size = 0

The Vector has space for 10 elements initially.

3.Creating Vector with Initial Values:

We can add values after creating the Vector.

Example:

```
Vector<String> names = new Vector<>();  
names.add("Ali");  
names.add("Sara");  
names.add("Omar");  
System.out.println(names);
```

Output:

```
[Ali, Sara, Omar]
```

07*Adding Elements to Vector:

Vector provides methods to insert new elements.

add() :

The add() method adds an element at the end of the Vector.

Syntax:

```
vectorName.add(element);
```

Example:

```
Vector<Integer> numbers = new Vector<>();  
numbers.add(10);  
numbers.add(20);  
numbers.add(30);  
System.out.println(numbers);
```

Output:

```
[10, 20, 30]
```

The order of elements is maintained.

add(index, element):

The add() method can also insert an element at a specific index.

Syntax:

```
vectorName.add(index, element);
```

Example:

```
Vector<String> names = new Vector<>();  
names.add("Ali");  
names.add("Ahmad");  
names.add("Sara");  
names.add(1,"Omar");  
System.out.println(names);
```

Output:

```
[Ali, Omar, Ahmad, Sara]
```

08*Accessing Vector Elements:

To access elements inside a Vector, we use the get() method.

Syntax:

```
vectorName.get(index);
```

Example:

```
Vector<Integer> numbers = new Vector<>();  
numbers.add(100);  
numbers.add(200);  
numbers.add(300);  
System.out.println(numbers.get(1));
```

Output:

```
200
```

09*Updating Vector Elements:

To change an existing element inside a Vector, we use the set() method.

The set() method replaces the value at a specific index.

Syntax:

```
vectorName.set(index, newValue);
```

Example:

```
import java.util.Vector;  
public class Main {  
    public static void main(String[] args) {  
        Vector<Integer> numbers = new Vector<>();  
        numbers.add(10);  
        numbers.add(20);  
        numbers.add(30);  
        System.out.println(numbers);  
        numbers.set(1, 50);  
        System.out.println(numbers);  
    }  
}
```

Output:

```
[10, 20, 30]
```

```
[10, 50, 30]
```

10*Removing Elements from Vector:

remove(index):

Removes an element using its index.

Syntax:

```
vectorName.remove(index);
```

Example:

```
Vector<Integer> numbers = new Vector<>();  
numbers.add(10);  
numbers.add(20);  
numbers.add(30);  
numbers.remove(1);  
System.out.println(numbers);
```

Output:

```
[10, 30]
```

remove(Object):

Removes a specific value from the Vector.

Example:

```
Vector<String> names = new Vector<>();
names.add("Ali");
names.add("Ahmad");
names.add("Sara");
names.remove("Ahmad");
System.out.println(names);
```

Output:

```
[Ali, Sara]
```

clear():

Removes all elements from the Vector.

Example:

```
Vector<Integer> numbers = new Vector<>();
numbers.add(10);
numbers.add(20);
numbers.clear();
System.out.println(numbers);
```

Output:

```
[]
```

11*Searching in Vector:

Vector provides methods to search for elements.

contains():

checks if an element exists in the Vector. (return true or false)

Syntax:

```
vectorName.contains(element);
```

Example:

```
Vector<String> names = new Vector<>();
names.add("Ali");
names.add("Sara");
System.out.println(names.contains("Ali"));
System.out.println(names.contains("Omar"));
```

Output:

```
true
false
```

indexOf() :

Returns the index of an element.

Syntax:

```
vectorName.indexOf(element);
```

Example:

```
Vector<Integer> numbers = new Vector<>();  
numbers.add(5);  
numbers.add(10);  
numbers.add(15);  
System.out.println(numbers.indexOf(10));
```

Output:

```
1
```

12* Vector Size and Capacity Methods:

Vector provides methods to get information about its storage.

size():

Returns the number of elements currently stored.

Example:

```
Vector<Integer> numbers = new Vector<>();  
numbers.add(10);  
numbers.add(20);  
numbers.add(30);  
System.out.println(numbers.size());
```

Output:

```
3
```

capacity():

Returns the total available storage capacity.

Example:

```
Vector<Integer> numbers = new Vector<>(10);  
System.out.println(numbers.capacity());
```

Output:

```
10
```

isEmpty():

Checks whether the Vector contains elements or not. (return true or false)

Example:

```
Vector<Integer> numbers = new Vector<>();  
System.out.println(numbers.isEmpty());  
numbers.add(5);  
System.out.println(numbers.isEmpty());
```

Output:

```
true  
false
```